

BỘ GIAO THÔNG VẬN TẢI
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN
MÔN: CÔNG NGHỆ PHẦN MỀM

XÂY DỰNG HỆ THỐNG CHATBOT
HỖ TRỢ HỌC TẬP ỨNG DỤNG
RETRIEVAL-AUGMENTED
GENERATION (RAG)

Giảng viên hướng dẫn: Thầy Nguyễn Văn Chiến

Nhóm thực hiện: Nhóm 9

- | | |
|--------------------------|--------------------|
| 1. Phan Liên Hưng | MSSV: 034206016528 |
| 2. Nguyễn Ninh Công Danh | MSSV: 079205006602 |
| 3. Nguyễn Hiên Đạt | MSSV: 079206026928 |
| 4. Đào Văn Khánh | MSSV: 051206009944 |
| 5. Phạm Hoàng Dũng | MSSV: 070205007005 |
| 6. Lương Thành Đạt | MSSV: 056206003075 |
| 7. Trần Thị Kiều Trinh | MSSV: 066306009531 |

Thành phố Hồ Chí Minh, tháng 7 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đề án “**Xây dựng Hệ thống Chatbot Hỗ trợ Học tập ứng dụng Retrieval-Augmented Generation (RAG)**”, nhóm chúng em đã nhận được rất nhiều sự hướng dẫn, giúp đỡ và tạo điều kiện thuận lợi từ phía Nhà trường, Khoa Công nghệ Thông tin cùng quý Thầy, Cô.

Đầu tiên, nhóm xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Nhà trường cùng toàn thể quý Thầy, Cô thuộc Khoa Công nghệ Thông tin đã truyền đạt những kiến thức nền tảng cũng như những kinh nghiệm quý báu trong suốt quá trình học tập. Đây chính là tiền đề quan trọng giúp nhóm có đủ kiến thức để hoàn thành đề án.

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc đến **Giảng viên hướng dẫn – Thầy Nguyễn Văn Chiến** đã tận tình hướng dẫn, góp ý và hỗ trợ nhóm trong suốt quá trình nghiên cứu, phân tích yêu cầu, thiết kế hệ thống, triển khai chương trình cũng như hoàn thiện báo cáo. Những góp ý của Thầy đã giúp nhóm hoàn thiện sản phẩm cả về mặt kỹ thuật lẫn nội dung báo cáo.

Trong quá trình thực hiện, nhóm đã tìm hiểu và ứng dụng nhiều công nghệ hiện đại như Retrieval-Augmented Generation (RAG), ChromaDB, FastAPI, ReactJS, mô hình ngôn ngữ lớn (LLM), kỹ thuật Embedding văn bản và xây dựng API phục vụ hệ thống Chatbot. Thông qua quá trình này, nhóm không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện kỹ năng nghiên cứu tài liệu, làm việc nhóm, quản lý tiến độ bằng Jira và quản lý mã nguồn bằng GitHub.

Mặc dù nhóm đã cố gắng hoàn thành đề án một cách tốt nhất, tuy nhiên do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế nên báo cáo chắc chắn vẫn còn nhiều thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy, Cô để có thể tiếp tục hoàn thiện sản phẩm trong tương lai.

Cuối cùng, nhóm xin kính chúc quý Thầy, Cô luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Thành phố Hồ Chí Minh, tháng 7 năm 2026

Nhóm 9 – Nhóm thực hiện

CONTENTS

Lời cảm ơn	i
1 GIỚI THIỆU DỰ ÁN	1
1.1 Bối cảnh và Động lực	1
1.2 Mục tiêu Dự án	1
1.3 Phạm vi Hệ thống	2
1.3.1 Phạm vi trong phạm vi nghiên cứu (In-scope)	2
1.3.2 Ngoài phạm vi nghiên cứu (Out-of-scope)	2
1.4 Đối tượng Người dùng (System Actors)	2
1.5 Phương pháp Nghiên cứu	3
1.6 Cấu trúc Báo cáo	3
1.7 Cơ sở Lý thuyết và Công nghệ Sử dụng	4
1.7.1 Retrieval-Augmented Generation (RAG)	4
1.7.2 Vector Database – ChromaDB	4
1.7.3 Backend – FastAPI	5
1.7.4 Frontend – ReactJS	5
1.7.5 Tiêu chuẩn Đánh giá – RAGAS	5
2 SWP – KẾ HOẠCH QUẢN LÝ DỰ ÁN PHẦN MỀM	6
2.1 Tổng quan Kế hoạch Dự án	6
2.2 Phạm vi Dự án	6
2.2.1 Phát biểu Phạm vi	6
2.2.2 Sản phẩm Bàn giao (Deliverables)	7
2.3 Ước lượng Công sức và Tài nguyên	7
2.3.1 Ước lượng Công sức	7
2.3.2 Phân công Nhân sự (RACI Matrix)	8
2.4 Quy trình Phát triển – Agile Scrum	8
2.4.1 Giới thiệu Agile Scrum	8
2.4.2 Quy trình Làm việc	8
2.4.3 Kế hoạch Sprint	9
2.5 Quản lý Backlog	9
2.5.1 Danh sách Epic	9
2.5.2 Một số User Story tiêu biểu	10
2.5.3 Minh họa Product Backlog trên Jira	10
2.5.4 Bảng Kanban Quản lý Công việc	13

2.6	Phân tích Rủi ro	14
2.7	Quản lý Cấu hình – GitHub	14
2.7.1	Chiến lược Nhánh (Branch Strategy)	14
2.7.2	Cấu trúc Repository	15
2.7.3	Lịch sử Commit	16
2.8	Tiêu chuẩn Lập trình	17
2.9	Kết luận Chương	17
3	SWR – ĐẶC TẢ YÊU CẦU PHẦN MỀM	18
3.1	Tổng quan Tài liệu SWR	18
3.2	Tổng quan Sản phẩm (Product Overview)	18
3.2.1	Biểu đồ Ngữ cảnh (Context Diagram)	18
3.2.2	Môi trường Vận hành (Operational Environment)	19
3.3	Yêu cầu Người dùng (User Requirements)	20
3.3.1	Mô tả Tác nhân (System Actors)	20
3.3.2	Biểu đồ Use Case Tổng quan	20
3.4	Yêu cầu Chức năng (Functional Requirements)	21
3.4.1	FR-01: Chức năng Chat RAG (Hỏi đáp bằng AI)	21
3.4.2	FR-02: Chức năng Tải lên Tài liệu (Document Upload & Ingestion)	22
3.4.3	FR-03: Chức năng Quản lý Tài liệu (Document Management)	23
3.4.4	FR-04: Chức năng Chat Sandbox (Cấu hình RAG)	24
3.4.5	FR-05: Chức năng Benchmark	25
3.4.6	FR-06: Chức năng Dashboard Phân tích	26
3.5	Yêu cầu Phi Chức năng (Non-Functional Requirements)	27
3.5.1	Hiệu năng (Performance)	27
3.5.2	Độ chính xác (Accuracy & Faithfulness)	27
3.5.3	Bảo mật (Security)	28
3.5.4	Khả năng Mở rộng (Scalability)	28
3.5.5	Khả dụng (Availability)	28
3.5.6	Khả năng Bảo trì (Maintainability)	28
3.6	Yêu cầu Giao diện Ngoài (External Interface Requirements)	28
3.6.1	Giao diện Người dùng (User Interface)	28
3.6.2	Giao diện Phần mềm (Software Interface)	29
3.7	Phụ lục Yêu cầu (Requirements Appendix)	29
3.7.1	Danh sách Thông báo Hệ thống (Application Messages List)	29
3.7.2	Thuật ngữ Giải thích (Glossary)	30
3.8	Kết luận Chương	30

4	SWD – MÔ TẢ THIẾT KẾ PHẦN MỀM	31
4.1	Tổng quan Thiết kế	31
4.2	Thiết kế Kiến trúc Hệ thống (System Architecture)	31
4.2.1	Kiến trúc Tổng thể	31
4.2.2	Package Diagram	32
4.3	Thiết kế Cơ sở Dữ liệu (Database Design)	32
4.3.1	Entity Relationship Diagram (ERD)	32
4.3.2	Mô tả Các Bảng Dữ liệu Chính	33
4.4	Thiết kế Chi tiết (Detailed Design)	36
4.4.1	Biểu đồ Lớp (Class Diagram)	36
4.4.2	Biểu đồ Trạng thái (State Machine Diagram)	38
4.4.2.1	State Machine: Vòng đời Tài liệu (Document Lifecycle)	38
4.4.2.2	State Machine: Vòng đời Phiên Hội thoại (Chat Session)	39
4.4.3	Biểu đồ Luồng (Flowchart)	40
4.4.3.1	Flowchart: RAG Pipeline (Luồng Hỏi đáp)	40
4.4.3.2	Flowchart: Ingestion Pipeline (Luồng Xử lý Tài liệu)	41
4.4.3.3	Flowchart: Benchmark Pipeline	42
4.4.4	Biểu đồ Hoạt động (Activity Diagram)	43
4.4.4.1	Activity Diagram: Tương tác Chat giữa Sinh viên và Hệ thống	43
4.4.4.2	Activity Diagram: Quản trị viên Upload và Quản lý Tài liệu	44
4.4.4.3	Activity Diagram: Quy trình Benchmark	45
4.4.5	Biểu đồ Use Case Chi tiết (Detailed Use Case)	46
4.4.5.1	UC-01: Đặt câu hỏi Chat RAG	46
4.4.5.2	UC-08: Chạy Benchmark	47
4.5	Thiết kế Giao diện Người dùng (UI Design)	47
4.5.1	Giao diện Trang chủ	47
4.5.2	Giao diện Chat Sandbox	48
4.6	Kết luận Chương	48
5	SWT – TÀI LIỆU KIỂM THỬ PHẦN MỀM	49
5.1	Tổng quan Tài liệu Kiểm thử	49
5.2	Phạm vi Kiểm thử (Testing Scope)	49
5.2.1	Trong phạm vi (In-scope)	49
5.2.2	Ngoài phạm vi (Out-of-scope)	49
5.3	Chiến lược Kiểm thử (Testing Strategy)	50
5.3.1	Mức độ Kiểm thử (Testing Levels)	50
5.3.2	Loại Kiểm thử (Testing Types)	50
5.4	Kế hoạch Kiểm thử (Test Plan)	50

5.4.1	Bảng Test Cases – Kiểm thử Chức năng	50
5.5	Môi trường Kiểm thử Benchmark (RAGAS)	52
5.5.1	Cấu hình Môi trường	52
5.5.2	Tập Dữ liệu Kiểm thử (Test Dataset)	53
5.6	Kết quả Kiểm thử (Test Reports)	53
5.6.1	Kết quả RAGAS Benchmark	53
5.6.2	Bảng So sánh Kết quả 4 Cấu hình RAG	54
5.6.3	Phân tích Kết quả	55
5.6.3.1	Phân tích Chiến lược Phân mảnh (Chunking)	55
5.6.3.2	Phân tích Mô hình Embedding	55
5.6.3.3	Khuyến nghị Cấu hình Tối ưu	55
5.7	Tóm tắt Kết quả Kiểm thử	56
5.7.1	Đánh giá Tổng kết	56
5.8	Kết luận Chương	56
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	57
6.1	Kết quả Đạt được	57
6.2	Đánh giá Ưu và Nhược điểm	58
6.2.1	Ưu điểm Nổi bật	58
6.2.2	Hạn chế Còn tồn đọng	58
6.3	Định hướng Phát triển Tương lai	58
6.4	Tổng kết	59

LIST OF FIGURES

2.1	Product Backlog trên Jira (Phần 1)	10
2.2	Product Backlog trên Jira (Phần 2)	11
2.3	Product Backlog trên Jira (Phần 3)	11
2.4	Product Backlog trên Jira (Phần 4)	12
2.5	Product Backlog trên Jira (Phần 5)	12
2.6	Bảng Kanban quản lý công việc trên Jira	13
2.7	Repository dự án trên GitHub	15
2.8	Cấu trúc thư mục Backend hệ thống	15
2.9	Lịch sử Commit dự án trên GitHub (Phần 1)	16
2.10	Lịch sử Commit dự án trên GitHub (Phần 2)	16
3.1	Biểu đồ Ngữ cảnh (Context Diagram) của hệ thống	19
3.2	Biểu đồ Use Case tổng quan của hệ thống	20
3.3	Giao diện Quản lý Tài liệu (Document Management)	24
3.4	Giao diện Chat Sandbox với bảng điều khiển cấu hình RAG	25
3.5	Giao diện cấu hình và chạy Benchmark	26
3.6	Dashboard tổng quan các chỉ số RAGAS	27
3.7	Biểu đồ Radar và bảng xếp hạng các cấu hình RAG	27
4.1	Kiến trúc tổng thể hệ thống Chatbot RAG	31
4.2	Sơ đồ thực thể quan hệ (ERD) của hệ thống	32
4.3	Biểu đồ Lớp (Class Diagram) của hệ thống	36
4.4	State Machine Diagram: Vòng đời Tài liệu (Document Lifecycle)	38
4.5	State Machine Diagram: Vòng đời Phiên Hội thoại (Chat Session)	39
4.6	Flowchart: RAG Pipeline – Luồng xử lý câu hỏi hỏi đáp	40
4.7	Flowchart: Ingestion Pipeline – Luồng xử lý tài liệu	41
4.8	Flowchart: Benchmark Pipeline	42
4.9	Activity Diagram: Tương tác Chat giữa Sinh viên và Hệ thống	43
4.10	Activity Diagram: Quản trị viên Upload và Quản lý Tài liệu	44
4.11	Activity Diagram: Quy trình Benchmark đánh giá RAG	45
4.12	Giao diện Trang chủ hệ thống	47
4.13	Giao diện Chat Sandbox với bảng cấu hình RAG	48
5.1	Giao diện cấu hình và khởi chạy Benchmark	53
5.2	Dashboard tổng quan kết quả RAGAS sau Benchmark	54
5.3	Biểu đồ Radar và Bảng xếp hạng các cấu hình RAG	54

LIST OF TABLES

1.1	Danh sách tác nhân hệ thống	3
1.2	Cấu trúc báo cáo	4
2.1	Thông tin chung dự án	6
2.2	Danh sách sản phẩm bàn giao	7
2.3	Ước lượng công sức theo module	7
2.4	Ma trận phân công trách nhiệm (RACI)	8
2.5	Kế hoạch Sprint dự án	9
2.6	Danh sách User Story tiêu biểu	10
2.7	Ma trận phân tích rủi ro dự án	14
2.8	Tiêu chuẩn lập trình áp dụng	17
3.1	Thông tin tài liệu SWR	18
3.2	Môi trường vận hành hệ thống	19
3.3	Mô tả chi tiết các tác nhân hệ thống	20
3.4	Danh sách giao diện phần mềm bên ngoài	29
3.5	Danh sách thông báo hệ thống	29
3.6	Bảng thuật ngữ sử dụng trong tài liệu	30
4.1	Mô tả bảng users	33
4.2	Mô tả bảng documents	33
4.3	Mô tả bảng conversations và messages	34
4.4	Mô tả bảng experiments và evaluations	35
4.5	Mô tả các lớp trong Class Diagram	37
4.6	Mô tả trạng thái tài liệu	38
4.7	Đặc tả Use Case UC-01: Chat RAG	46
4.8	Đặc tả Use Case UC-08: Benchmark	47
5.1	Thông tin tài liệu SWT	49
5.2	Các mức độ kiểm thử áp dụng	50
5.3	Test Cases kiểm thử chức năng hệ thống	51
5.4	Cấu hình môi trường kiểm thử Benchmark	52
5.5	Phân loại câu hỏi trong Test Dataset	53
5.6	Kết quả Benchmark so sánh 4 cấu hình RAG (LLM cố định: Gemini 2.5 Flash)	54
5.7	Tóm tắt kết quả kiểm thử toàn bộ hệ thống	56

CHAPTER 1

GIỚI THIỆU DỰ ÁN

1.1 Bối cảnh và Động lực

Trong bối cảnh trí tuệ nhân tạo (Artificial Intelligence – AI) phát triển mạnh mẽ, các mô hình ngôn ngữ lớn (Large Language Models – LLM) đang được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, y tế, tài chính và dịch vụ khách hàng. Đặc biệt, các hệ thống Chatbot thông minh đã trở thành công cụ hỗ trợ hiệu quả trong việc tìm kiếm, tổng hợp và trả lời thông tin.

Trong lĩnh vực giáo dục, sinh viên thường gặp khó khăn khi phải tìm kiếm thông tin từ nhiều nguồn tài liệu khác nhau như giáo trình, slide bài giảng, tài liệu PDF hoặc tài liệu tham khảo. Việc đọc và tổng hợp lượng lớn dữ liệu mất nhiều thời gian và đôi khi không mang lại kết quả chính xác. Hiện nay, phần lớn sinh viên sử dụng các công cụ AI phổ biến như ChatGPT hoặc Gemini, tuy nhiên các mô hình này chủ yếu dựa trên dữ liệu tiền huấn luyện và không thể truy cập trực tiếp vào tài liệu nội bộ, giáo trình đặc thù của từng môn học.

Kỹ thuật **Retrieval-Augmented Generation (RAG)** nổi lên như một giải pháp đột phá, cho phép hệ thống tự động truy xuất trực tiếp các luồng thông tin từ tài liệu do người dùng cung cấp trước khi chuyển giao ngữ cảnh cho mô hình ngôn ngữ để sinh câu trả lời. Cơ chế này đảm bảo tính chính xác, khả năng trích dẫn minh bạch và giảm thiểu hiện tượng “ảo giác” (hallucination).

Xuất phát từ nhu cầu đó, nhóm thực hiện đề tài:

XÂY DỰNG HỆ THỐNG CHAT- BOT HỖ TRỢ HỌC TẬP ỨNG DỤNG RETRIEVAL- AUGMENTED GENERATION (RAG)

1.2 Mục tiêu Dự án

Đề tài hướng đến việc giải quyết các bài toán về truy xuất và tổng hợp tri thức với những mục tiêu cụ thể sau:

1. Xây dựng hệ thống Chatbot hoàn chỉnh có khả năng hỗ trợ trả lời các câu hỏi chuyên sâu liên quan trực tiếp đến tài liệu môn học của sinh viên.

2. Xây dựng quy trình xử lý tài liệu tự động (Ingestion Pipeline) bao gồm: đọc file PDF/DOCX, làm sạch văn bản, phân mảnh dữ liệu (Chunking), tạo vector (Embedding) và lưu trữ trong ChromaDB.
3. Tích hợp mô hình ngôn ngữ lớn (LLM – Gemini 2.5 Flash) thông qua FastAPI để sinh câu trả lời tự nhiên, chính xác dựa trên ngữ cảnh được cung cấp.
4. Xây dựng giao diện Web bằng ReactJS nhằm đem lại trải nghiệm tương tác mượt mà, thân thiện cho cả sinh viên và người quản trị.
5. Thiết lập module Benchmark để thử nghiệm, đo lường và so sánh hiệu năng giữa các cấu hình RAG khác nhau dựa trên bộ tiêu chí RAGAS (Accuracy, Relevancy, Faithfulness, Latency).

1.3 Phạm vi Hệ thống

1.3.1 Phạm vi trong phạm vi nghiên cứu (In-scope)

- Hỗ trợ truy vấn thông tin từ tài liệu văn bản tĩnh (PDF, DOCX).
- Triển khai kỹ thuật RAG kết hợp với cơ sở dữ liệu vector ChromaDB.
- Phát triển kiến trúc Client-Server: Backend FastAPI + Frontend ReactJS.
- Đánh giá hiệu năng các mô hình thông qua bộ tiêu chuẩn định lượng RAGAS.
- Quản lý dự án bằng Jira (Agile Scrum) và kiểm soát mã nguồn bằng GitHub.

1.3.2 Ngoài phạm vi nghiên cứu (Out-of-scope)

- Nhận dạng giọng nói và xử lý hình ảnh phức tạp.
- Hỗ trợ đa ngôn ngữ ở quy mô lớn (ngoài tiếng Việt và tiếng Anh).
- Tinh chỉnh mô hình sâu (Fine-Tuning LLM) do hạn chế tài nguyên phần cứng.
- Triển khai trên Cloud (AWS, GCP) – chỉ dừng ở môi trường cục bộ.

1.4 Đối tượng Người dùng (System Actors)

Hệ thống được thiết kế hướng tới hai nhóm người dùng chính:

Table 1.1: Danh sách tác nhân hệ thống

STT	Tác nhân	Mô tả
1	Sinh viên (Student)	Người dùng cuối, đặt câu hỏi, nhận câu trả lời từ Chatbot, xem lịch sử trò chuyện và cấu hình RAG trong Sandbox.
2	Quản trị viên (Admin)	Quản lý kho tài liệu (tải lên, xóa, cập nhật), theo dõi hệ thống, chạy Benchmark và xem Dashboard phân tích.
3	Mô hình AI (LLM)	Tác nhân bên ngoài (Gemini 2.5 Flash, Google Embedding, BAAI/bge-m3), sinh câu trả lời từ Prompt được bơm ngữ cảnh.
4	Vector Database	ChromaDB lưu trữ và truy xuất các Vector Embedding từ tài liệu.

1.5 Phương pháp Nghiên cứu

Quá trình thực hiện đề tài được diễn ra tuần tự qua các giai đoạn:

- Giai đoạn 1: Khảo sát và Phân tích:** Khảo sát thực trạng và các mô hình Chatbot hỗ trợ học tập hiện có. Nghiên cứu chuyên sâu về kiến trúc RAG và các kỹ thuật liên quan (Chunking, Vector Database, Prompt Engineering).
- Giai đoạn 2: Lập kế hoạch:** Thiết lập quy trình Agile Scrum trên Jira, phân công nhiệm vụ, ước lượng thời gian và xác định rủi ro.
- Giai đoạn 3: Thiết kế:** Thiết kế kiến trúc tổng thể, vẽ các biểu đồ UML (Use Case, Class Diagram, Activity, State Machine, Flowchart), thiết kế CSDL.
- Giai đoạn 4: Triển khai:** Lập trình Backend FastAPI, tích hợp ChromaDB và LLM API; lập trình Frontend ReactJS.
- Giai đoạn 5: Kiểm thử:** Xây dựng bộ test, kiểm thử chức năng và chạy Benchmark đánh giá hiệu năng theo chuẩn RAGAS.
- Giai đoạn 6: Tổng kết:** Tổng hợp số liệu, phân tích kết quả và hoàn thiện tài liệu báo cáo.

1.6 Cấu trúc Báo cáo

Tài liệu báo cáo đồ án được tổ chức theo chuẩn SWP/SWR/SWD/SWT như sau:

Table 1.2: Cấu trúc báo cáo

Chương	Tiêu đề	Nội dung
I	Giới thiệu Dự án	Bối cảnh, mục tiêu, phạm vi, đối tượng người dùng.
II	SWP – Kế hoạch Quản lý Dự án	Phạm vi, ước lượng, rủi ro, Sprint, phân công, GitHub.
III	SWR – Đặc tả Yêu cầu Phần mềm	Context Diagram, Use Case, Functional/Non-Functional Requirements, Business Rules.
IV	SWD – Thiết kế Phần mềm	Kiến trúc, Database Design, Use Case chi tiết, Class Diagram, State Machine, Flowchart, Activity Diagram.
V	SWT – Tài liệu Kiểm thử	Phạm vi, chiến lược, Test Cases, Benchmark RAGAS, báo cáo kết quả.
VI	Kết luận và Hướng phát triển	Kết quả đạt được, ưu/nhược điểm, hướng phát triển tương lai.

1.7 Cơ sở Lý thuyết và Công nghệ Sử dụng

1.7.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation là kiến trúc tiên tiến kết hợp sức mạnh của hệ thống truy xuất thông tin (Information Retrieval) và khả năng sinh văn bản của LLM. Thay vì chỉ dựa vào kiến thức tiền huấn luyện tĩnh, RAG cho phép hệ thống chủ động tìm kiếm và trích xuất tài liệu liên quan từ cơ sở dữ liệu động trước khi gửi ngữ cảnh vào mô hình AI.

Quy trình RAG gồm 3 giai đoạn chính:

- **Indexing (Lập chỉ mục):** Tài liệu được đọc, phân mảnh (Chunking) và chuyển thành Vector Embedding, lưu vào ChromaDB.
- **Retrieval (Truy xuất):** Câu hỏi của người dùng được mã hóa thành Vector, tìm kiếm các đoạn văn bản tương đồng nhất bằng Cosine Similarity.
- **Generation (Sinh kết quả):** Các đoạn văn bản trích xuất được ghép vào Prompt Template và gửi đến LLM để sinh câu trả lời cuối cùng.

1.7.2 Vector Database – ChromaDB

ChromaDB là hệ quản trị cơ sở dữ liệu Vector mã nguồn mở, được thiết kế chuyên biệt để lưu trữ và truy vấn tốc độ cao các dữ liệu Embedding. Trong hệ thống, ChromaDB đóng vai trò bộ nhớ dài hạn, lưu trữ hàng triệu Vector sinh ra từ tài liệu và thực hiện tìm kiếm tương đồng (Similarity Search) với độ trễ thấp.

1.7.3 Backend – FastAPI

FastAPI là Web Framework hiện đại, hiệu năng cao cho Python. Nổi bật với hỗ trợ lập trình bất đồng bộ (Async), tự động sinh tài liệu API (Swagger/Redoc), và tương thích cao với các thư viện AI/ML.

1.7.4 Frontend – ReactJS

ReactJS là thư viện JavaScript mã nguồn mở của Meta, xây dựng giao diện theo hướng Component. Cơ chế Virtual DOM giúp cập nhật và hiển thị dữ liệu theo thời gian thực vô cùng mượt mà.

1.7.5 Tiêu chuẩn Đánh giá – RAGAS

RAGAS (Retrieval Augmented Generation Assessment) là bộ tiêu chuẩn đánh giá chất lượng hệ thống RAG với 4 chỉ số chính:

- **Accuracy (Độ chính xác):** Tỷ lệ câu trả lời đúng so với đáp án chuẩn.
- **Relevancy (Độ bám sát):** Câu trả lời có giải quyết đúng trọng tâm câu hỏi.
- **Faithfulness (Độ trung thực):** Tỷ lệ thông tin được trích xuất hoàn toàn từ tài liệu nguồn.
- **Latency (Độ trễ):** Thời gian trung bình xử lý toàn bộ chu trình RAG.

CHAPTER 2

SWP – KẾ HOẠCH QUẢN LÝ DỰ ÁN PHẦN MỀM

2.1 Tổng quan Kế hoạch Dự án

Chương này trình bày **Kế hoạch Quản lý Dự án Phần mềm (Software Project Management Plan – SWP)** cho hệ thống Chatbot hỗ trợ học tập ứng dụng RAG. Tài liệu này bao gồm phạm vi dự án, ước lượng công sức, phân tích rủi ro, kế hoạch tiến độ, quy trình phát triển và quản lý cấu hình mã nguồn.

Table 2.1: Thông tin chung dự án

Thông tin	Chi tiết
Tên dự án	Xây dựng Hệ thống Chatbot Hỗ trợ Học tập ứng dụng RAG
Môn học	Công nghệ Phần mềm
Giảng viên hướng dẫn	Thầy Nguyễn Văn Chiến
Nhóm thực hiện	Nhóm 9 (7 thành viên)
Thời gian thực hiện	Tháng 3 – Tháng 7 năm 2026
Phương pháp quản lý	Agile Scrum (8 Sprint, 2 tuần/Sprint)
Công cụ quản lý	Jira Software, GitHub

2.2 Phạm vi Dự án

2.2.1 Phát biểu Phạm vi

Dự án xây dựng một hệ thống Chatbot thông minh ứng dụng kỹ thuật RAG phục vụ sinh viên và quản trị viên trong môi trường học tập. Hệ thống bao gồm:

- **Frontend:** Giao diện ReactJS với Chat Sandbox, quản lý tài liệu và Dashboard Benchmark.
- **Backend:** REST API FastAPI xử lý toàn bộ nghiệp vụ RAG, Embedding và LLM.
- **AI Pipeline:** Ingestion Pipeline (Chunking + Embedding) và RAG Pipeline (Retrieval + Generation).
- **Database:** ChromaDB (Vector) + MySQL (Quan hệ).
- **Benchmark Module:** Đánh giá định lượng các cấu hình RAG theo RAGAS.

2.2.2 Sản phẩm Bàn giao (Deliverables)

Table 2.2: Danh sách sản phẩm bàn giao

STT	Sản phẩm	Mô tả	Thời hạn
1	SWP Document	Kế hoạch quản lý dự án	Sprint 1
2	SWR Document	Đặc tả yêu cầu phần mềm	Sprint 2
3	SWD Document	Tài liệu thiết kế phần mềm	Sprint 3
4	Source Code	Backend FastAPI + Frontend ReactJS	Sprint 6
5	SWT Document	Báo cáo kiểm thử + Benchmark	Sprint 7
6	Final Report	Báo cáo tổng hợp LaTeX	Sprint 8

2.3 Ước lượng Công sức và Tài nguyên

2.3.1 Ước lượng Công sức

Table 2.3: Ước lượng công sức theo module

STT	Module	Công việc chính	Người	Tuần
1	Backend FastAPI	API design, RAG pipeline, Auth	2	6
2	Frontend ReactJS	UI/UX, Chat, Dashboard	2	6
3	AI/ML Pipeline	Embedding, ChromaDB, LLM	2	5
4	Benchmark Module	Test framework, RAGAS metrics	1	4
5	Database	MySQL schema, ChromaDB setup	1	3
6	Testing	Test cases, kiểm thử tích hợp	2	3
7	Documentation	SWP/SWR/SWD/SWT + báo cáo	2	4

2.3.2 Phân công Nhân sự (RACI Matrix)

Table 2.4: Ma trận phân công trách nhiệm (RACI)

Công việc	Hưng	Danh	Đạt H	Khánh	Dũng	Đạt L	Trinh
Thiết kế hệ thống	R	C	C	I	I	I	I
Backend FastAPI	R	A	C	I	C	I	I
Frontend ReactJS	I	I	R	A	C	I	C
RAG Pipeline	A	R	I	C	I	C	I
ChromaDB & DB	I	A	I	R	I	C	I
Benchmark Module	C	I	I	I	R	A	I
Kiểm thử	I	I	C	C	I	I	R
Tài liệu báo cáo	A	C	I	I	I	C	R

R=Responsible, A=Accountable, C=Consulted, I=Informed

2.4 Quy trình Phát triển – Agile Scrum

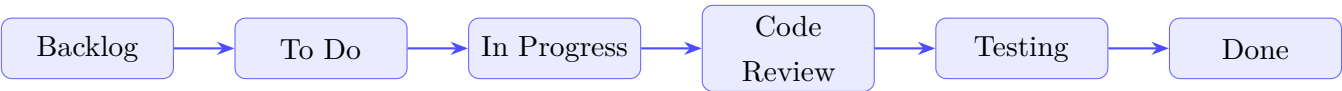
2.4.1 Giới thiệu Agile Scrum

Nhóm áp dụng phương pháp Agile Scrum với các đặc điểm:

- **Sprint:** Chu kỳ phát triển 2 tuần/sprint, tổng 8 sprint (16 tuần).
- **Daily Standup:** Họp ngắn hàng ngày báo cáo tiến độ.
- **Sprint Review:** Kiểm tra sản phẩm cuối sprint với giảng viên.
- **Sprint Retrospective:** Rút kinh nghiệm để cải thiện quy trình.
- **Product Backlog:** Toàn bộ User Story được quản lý trên Jira.

2.4.2 Quy trình Làm việc

Mỗi công việc trên Jira trải qua các trạng thái:



2.4.3 Kế hoạch Sprint

Table 2.5: Kế hoạch Sprint dự án

Sprint	Nội dung chính	Thời gian	Trạng thái
Sprint 1	Phân tích yêu cầu, thiết kế kiến trúc hệ thống, viết SWP/SWR	Tuần 1–2	Hoàn thành
Sprint 2	Thiết kế CSDL, vẽ biểu đồ UML, viết SWD	Tuần 3–4	Hoàn thành
Sprint 3	Xây dựng Backend FastAPI (CRUD, Auth, API cơ bản)	Tuần 5–6	Hoàn thành
Sprint 4	Xây dựng Frontend ReactJS (Chat UI, Upload)	Tuần 7–8	Hoàn thành
Sprint 5	Tích hợp ChromaDB, RAG Pipeline, LLM API	Tuần 9–10	Hoàn thành
Sprint 6	Module Benchmark, Dashboard, tích hợp đầy đủ	Tuần 11–12	Hoàn thành
Sprint 7	Kiểm thử hệ thống, chạy Benchmark RAGAS	Tuần 13–14	Hoàn thành
Sprint 8	Hoàn thiện báo cáo LaTeX, review và submit	Tuần 15–16	Hoàn thành

2.5 Quản lý Backlog

2.5.1 Danh sách Epic

Các Epic chính của dự án bao gồm:

- **EP-01:** Xây dựng Backend FastAPI
- **EP-02:** Xây dựng Frontend ReactJS
- **EP-03:** Xây dựng AI/RAG Pipeline
- **EP-04:** Tích hợp ChromaDB & MySQL
- **EP-05:** Module Benchmark & RAGAS
- **EP-06:** Thiết kế giao diện & UX
- **EP-07:** Kiểm thử hệ thống
- **EP-08:** Viết tài liệu báo cáo

2.5.2 Một số User Story tiêu biểu

Table 2.6: Danh sách User Story tiêu biểu

ID	Vai trò	User Story	Story Points
US-01	Sinh viên	Đặt câu hỏi và nhận câu trả lời từ Chatbot dựa trên tài liệu môn học.	8
US-02	Sinh viên	Xem lịch sử hội thoại và tiếp tục ngữ cảnh trò chuyện.	5
US-03	Sinh viên	Cấu hình thông số RAG (Top-K, Chunking, Embedding) trong Sandbox.	8
US-04	Quản trị viên	Tải lên tài liệu PDF/DOCX và theo dõi trạng thái xử lý.	8
US-05	Quản trị viên	Quản lý, xóa, cập nhật tài liệu trong kho tri thức.	5
US-06	Quản trị viên	Chạy Benchmark với nhiều cấu hình RAG khác nhau.	13
US-07	Quản trị viên	Xem Dashboard tổng quan hiệu năng qua biểu đồ Radar.	8
US-08	Hệ thống	Tự động phân mảnh và tạo Vector Embedding khi tài liệu được tải lên.	13

2.5.3 Minh họa Product Backlog trên Jira

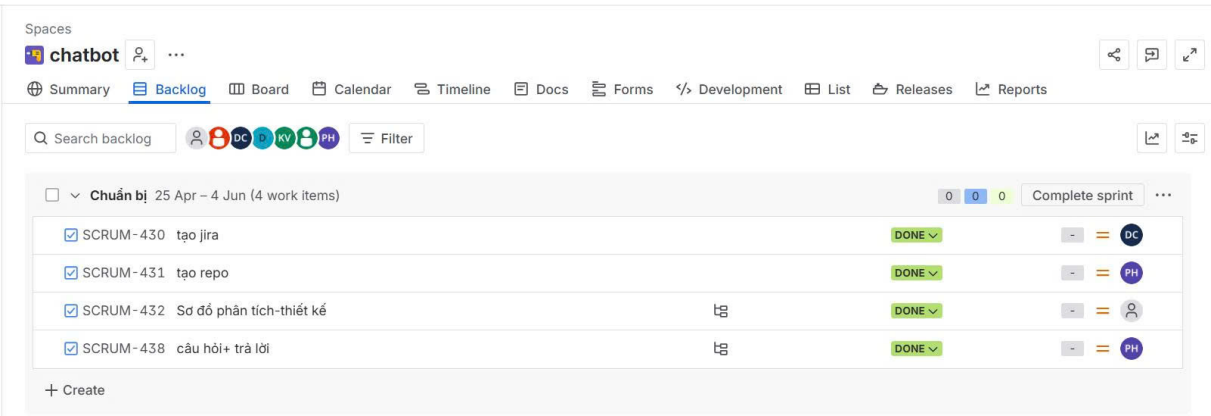


Figure 2.1: Product Backlog trên Jira (Phần 1)

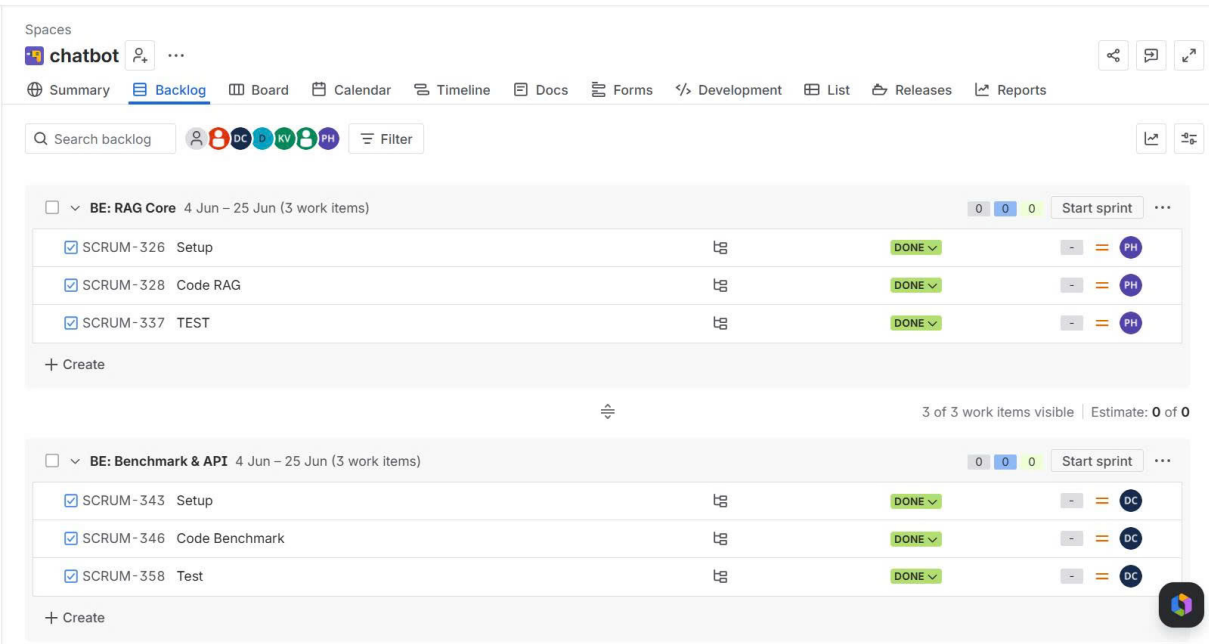


Figure 2.2: Product Backlog trên Jira (Phần 2)

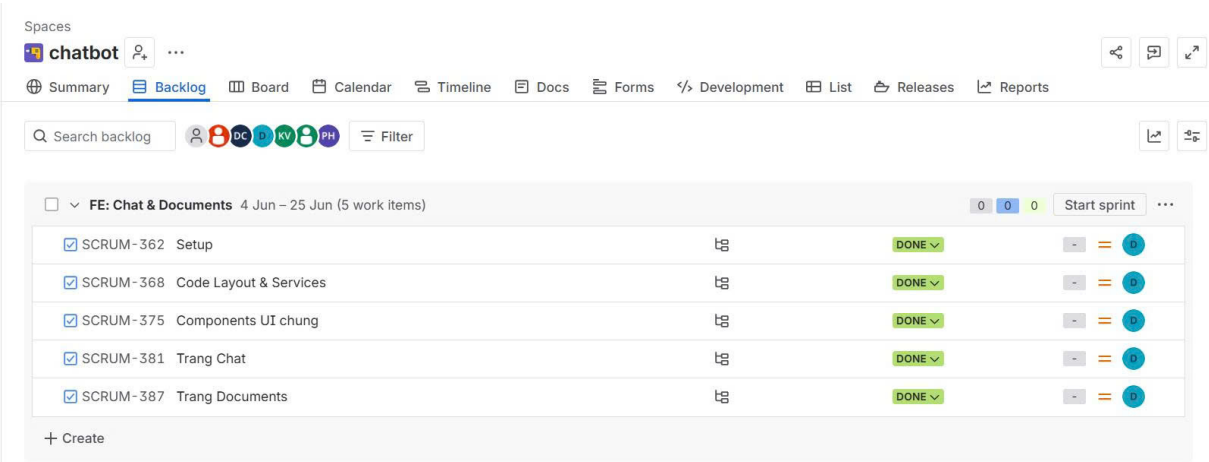


Figure 2.3: Product Backlog trên Jira (Phần 3)

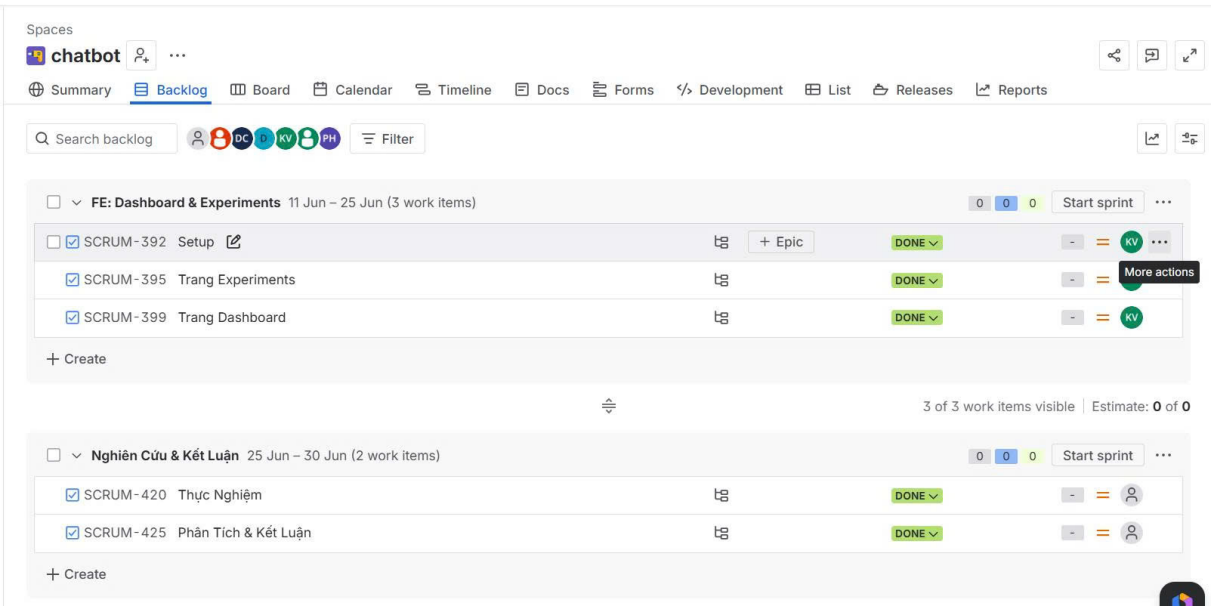


Figure 2.4: Product Backlog trên Jira (Phần 4)

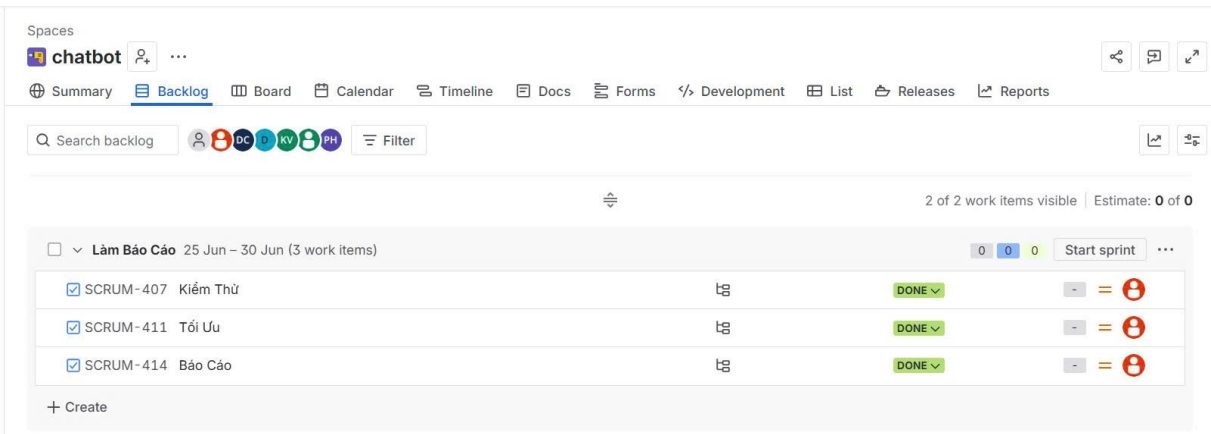


Figure 2.5: Product Backlog trên Jira (Phần 5)

2.5.4 Bảng Kanban Quản lý Công việc

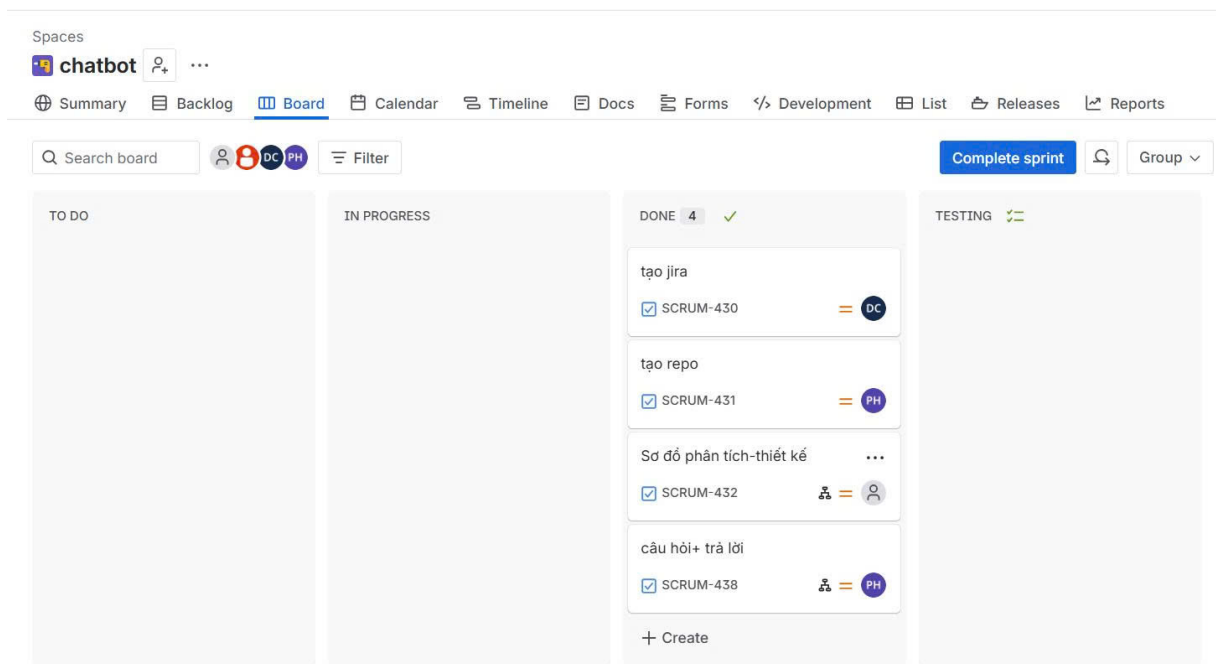


Figure 2.6: Bảng Kanban quản lý công việc trên Jira

Hình 2.6 thể hiện toàn bộ bảng Kanban của dự án. Mỗi thẻ (Card) đại diện cho một nhiệm vụ cụ thể và được di chuyển giữa các cột tương ứng với trạng thái công việc trong suốt vòng đời phát triển phần mềm.

2.6 Phân tích Rủi ro

Table 2.7: Ma trận phân tích rủi ro dự án

ID	Rủi ro	Xác suất	Tác động	Biện pháp giảm thiểu
R01	API LLM (Gemini) bị giới hạn rate limit	Cao	Cao	Implement retry logic và caching kết quả.
R02	Hiệu suất ChromaDB chậm với lượng tài liệu lớn	Trung bình	Cao	Tối ưu chunking size, sử dụng batch embedding.
R03	Thành viên nhóm không đồng bộ tiến độ	Cao	Trung bình	Scrum daily standup, review code pull request.
R04	Kết quả Benchmark không ổn định	Trung bình	Trung bình	Chạy benchmark nhiều lần, lấy trung bình.
R05	Tài liệu tải lên có format phức tạp (bảng, hình)	Cao	Trung bình	Sử dụng pdfplumber kết hợp OCR fallback.
R06	Xung đột code khi merge	Trung bình	Thấp	Branch strategy rõ ràng, PR review bắt buộc.

2.7 Quản lý Cấu hình – GitHub

2.7.1 Chiến lược Nhánh (Branch Strategy)

Nhóm áp dụng chiến lược nhánh **Git Flow** với cấu trúc:

- **main**: Nhánh chính, chỉ nhận code đã được review và test đầy đủ.
- **develop**: Nhánh tích hợp, tổng hợp code từ các feature branch.
- **feature/[tên-feature]**: Nhánh phát triển tính năng (ví dụ: **feature/rag-pipeline**).
- **hotfix/[tên-fix]**: Nhánh sửa lỗi khẩn cấp.

Quy trình: **feature branch** → **Pull Request** → **Code Review** (tối thiểu 1 reviewer) → **develop** → **main**.

2.7.2 Cấu trúc Repository

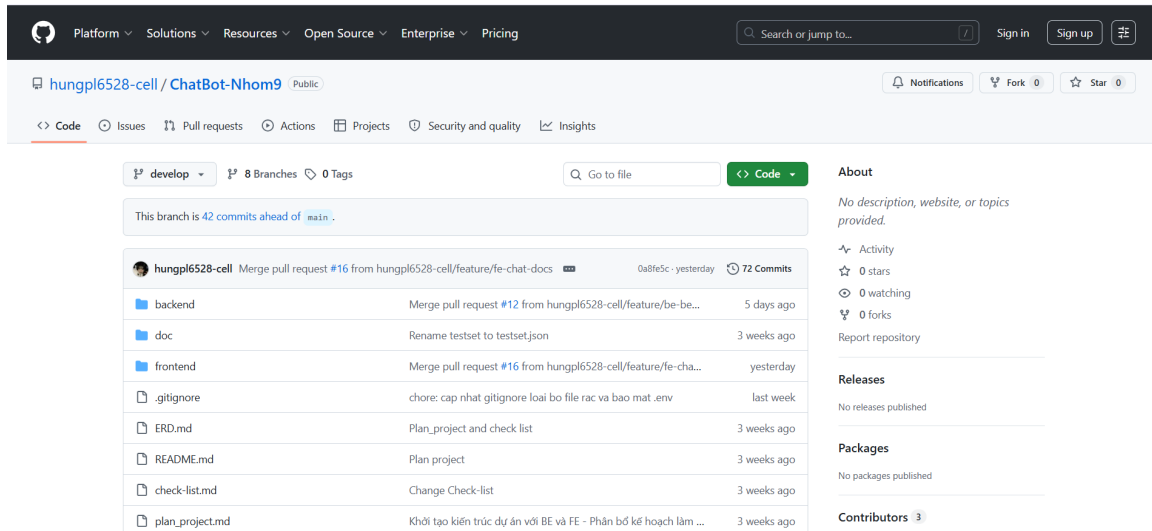


Figure 2.7: Repository dự án trên GitHub

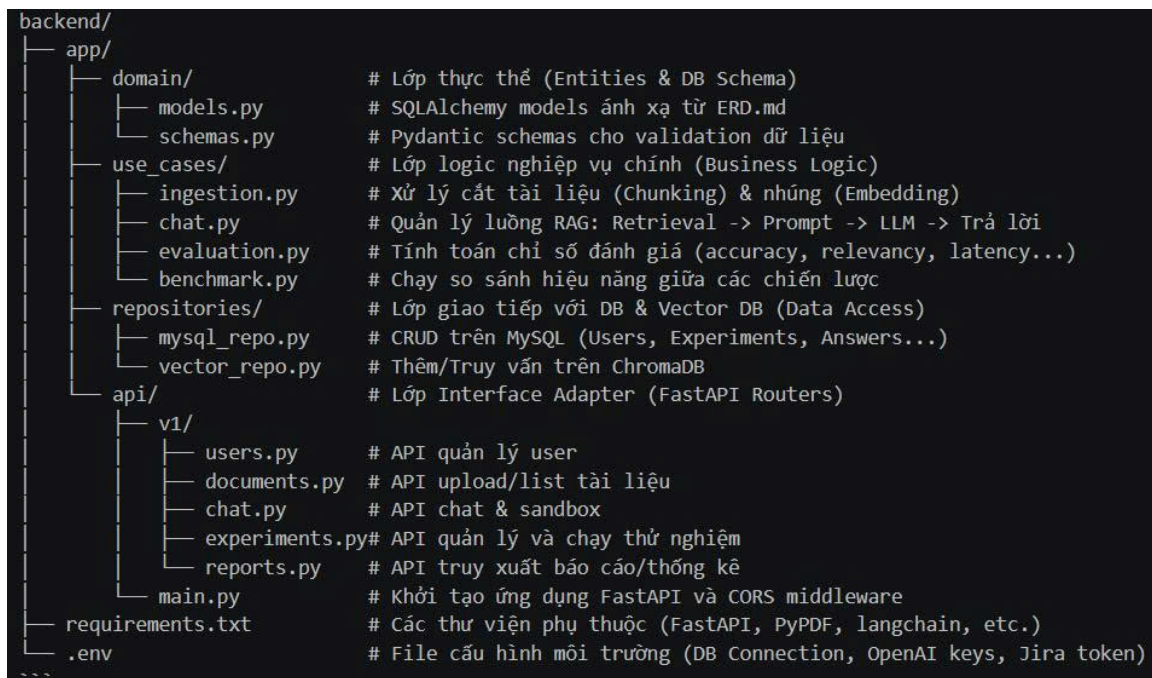


Figure 2.8: Cấu trúc thư mục Backend hệ thống

Repository được tổ chức thành các thư mục chính:

- /backend – Mã nguồn FastAPI, các module RAG, Benchmark.
- /frontend – Mã nguồn ReactJS, các component giao diện.
- /doc – Tài liệu dự án (SWP, SWR, SWD, SWT), báo cáo LaTeX.
- /chromadb_data – Dữ liệu Vector Database.

- README.md – Hướng dẫn cài đặt và chạy dự án.

2.7.3 Lịch sử Commit

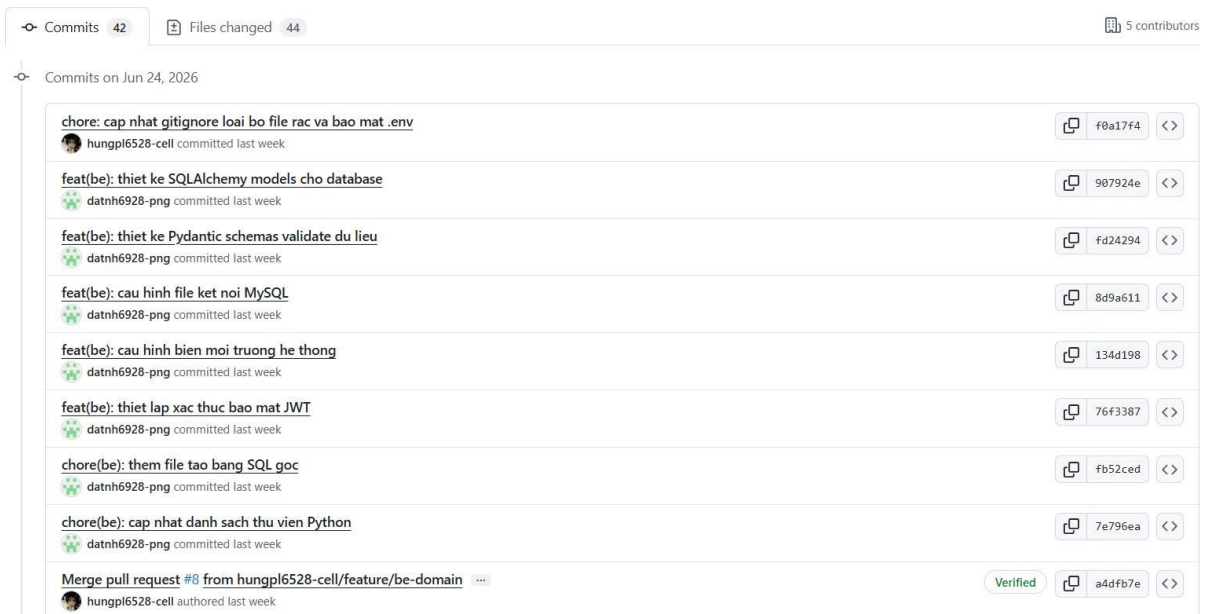


Figure 2.9: Lịch sử Commit dự án trên GitHub (Phần 1)

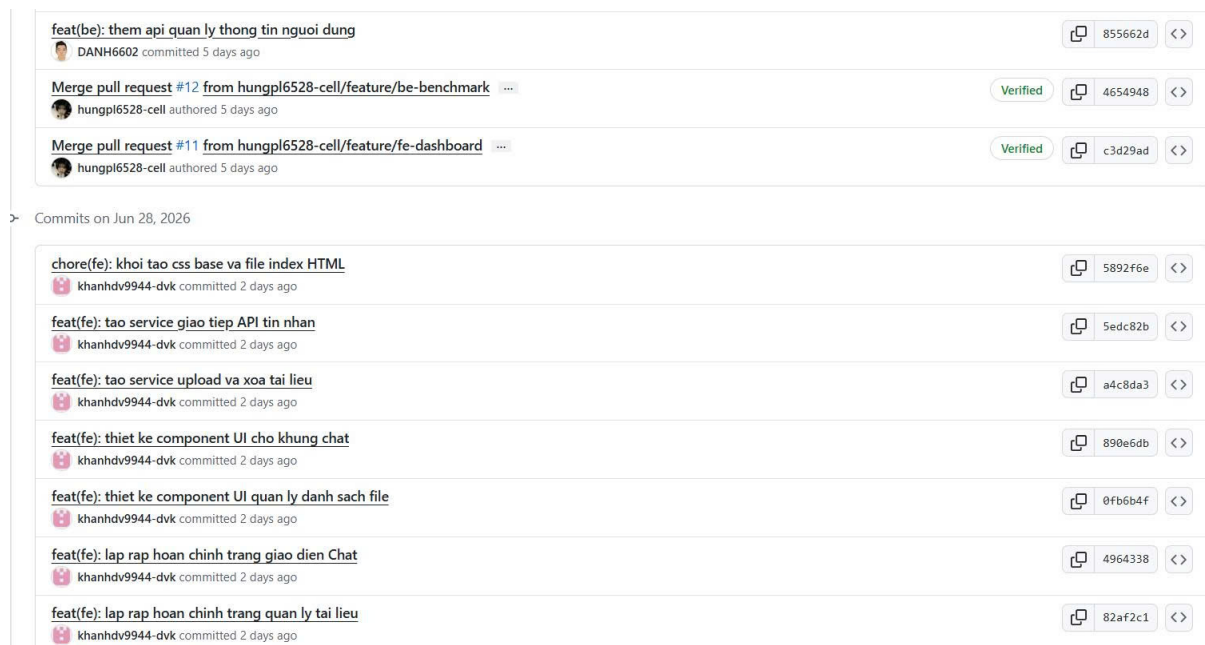


Figure 2.10: Lịch sử Commit dự án trên GitHub (Phần 2)

2.8 Tiêu chuẩn Lập trình

Table 2.8: Tiêu chuẩn lập trình áp dụng

Ngôn ngữ	Tiêu chuẩn	Quy định
Python	PEP 8	Snake_case, docstring bắt buộc, max 120 ký tự/dòng.
JavaScript	ESLint Airbnb	CamelCase, arrow functions, async/await.
SQL	SQL Standard	UPPER CASE cho từ khóa, snake_case cho tên cột.
Git Commit	Conventional Commits	feat/fix/docs/test: mô tả ngắn gọn bằng tiếng Anh.

2.9 Kết luận Chương

Chương này đã trình bày đầy đủ kế hoạch quản lý dự án phần mềm (SWP) bao gồm: phạm vi và sản phẩm bàn giao, ước lượng công sức, phân công trách nhiệm (RACI), kế hoạch Sprint theo Agile Scrum, phân tích rủi ro và chiến lược quản lý cấu hình bằng GitHub. Đây là nền tảng để nhóm triển khai dự án một cách có hệ thống, đúng tiến độ và đảm bảo chất lượng.

CHAPTER 3

SWR – ĐẶC TẢ YÊU CẦU PHẦN MỀM

3.1 Tổng quan Tài liệu SWR

Tài liệu **Đặc tả Yêu cầu Phần mềm (Software Requirements Specification – SWR)** mô tả đầy đủ và chi tiết các yêu cầu đối với hệ thống Chatbot hỗ trợ học tập ứng dụng RAG. Tài liệu này là cơ sở cốt lõi để nhóm phát triển tiến hành thiết kế kiến trúc, lập trình, kiểm thử và đánh giá mức độ hoàn thiện của hệ thống.

Table 3.1: Thông tin tài liệu SWR

Mục	Nội dung
Phiên bản	1.0
Ngày tạo	07/2026
Đối tượng áp dụng	Đội phát triển, giảng viên, khách hàng
Phạm vi	Hệ thống Chatbot RAG – Nhóm 9

3.2 Tổng quan Sản phẩm (Product Overview)

3.2.1 Biểu đồ Ngữ cảnh (Context Diagram)

Biểu đồ Context mô tả hệ thống ở mức độ trừu tượng cao nhất, thể hiện các tác nhân bên ngoài và các luồng dữ liệu chính trao đổi với hệ thống Chatbot.

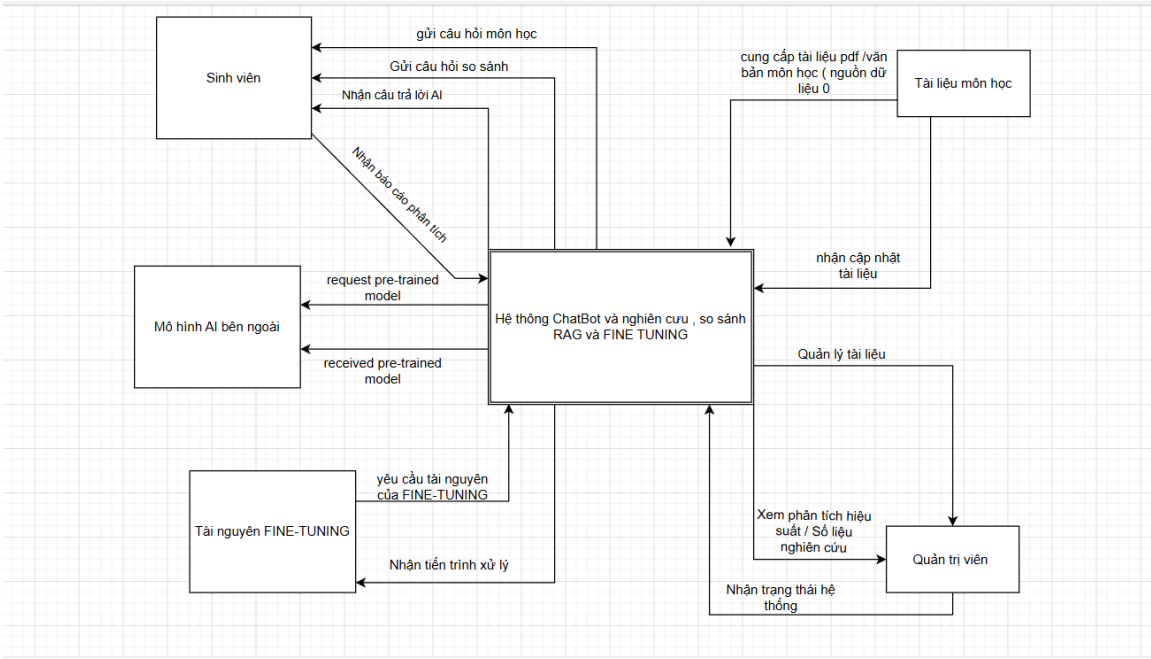


Figure 3.1: Biểu đồ Ngữ cảnh (Context Diagram) của hệ thống

Trong biểu đồ Context, hệ thống Chatbot (Backend FastAPI) đóng vai trò trung tâm:

- **Sinh viên** ↔ Hệ thống: Gửi câu hỏi, nhận câu trả lời kèm trích dẫn nguồn.
- **Quản trị viên** ↔ Hệ thống: Tải lên tài liệu, quản lý kho tri thức, xem Dashboard.
- **Mô hình AI (LLM)** ↔ Hệ thống: Nhận Prompt, trả về câu trả lời sinh ngôn ngữ tự nhiên.
- **ChromaDB** ↔ Hệ thống: Lưu trữ và truy xuất Vector Embedding.

3.2.2 Môi trường Vận hành (Operational Environment)

Table 3.2: Môi trường vận hành hệ thống

Thành phần	Yêu cầu
Hệ điều hành	Windows 10+, Ubuntu 20.04+, macOS 12+
Python	3.10 trở lên
Node.js	18.x trở lên
RAM	Tối thiểu 8GB (đề xuất 16GB)
GPU (tùy chọn)	Cho phép chạy mô hình Embedding cục bộ (BAAI/bge-m3)
Kết nối mạng	Bắt buộc để gọi Gemini API
Trình duyệt	Chrome 100+, Firefox 100+, Edge 100+

3.3 Yêu cầu Người dùng (User Requirements)

3.3.1 Mô tả Tác nhân (System Actors)

Table 3.3: Mô tả chi tiết các tác nhân hệ thống

Tác nhân	Đặc điểm	Mục tiêu sử dụng
Sinh viên	Người dùng cuối, không cần kiến thức kỹ thuật sâu; truy cập qua trình duyệt web.	Đặt câu hỏi nhanh về nội dung môn học, xem lịch sử hội thoại, thử nghiệm cấu hình RAG.
Quản trị viên	Người có quyền quản trị cao; am hiểu về tài liệu môn học và cấu hình hệ thống.	Quản lý kho tài liệu, theo dõi hoạt động hệ thống, chạy Benchmark đánh giá chất lượng.
LLM (AI Agent)	Tác nhân phần mềm bên ngoài (Gemini API) xử lý ngôn ngữ tự nhiên.	Sinh câu trả lời dựa trên Prompt được cung cấp bởi Backend.
ChromaDB	Cơ sở dữ liệu Vector nội bộ.	Lưu trữ và truy vấn Vector Embedding với tốc độ cao.

3.3.2 Biểu đồ Use Case Tổng quan

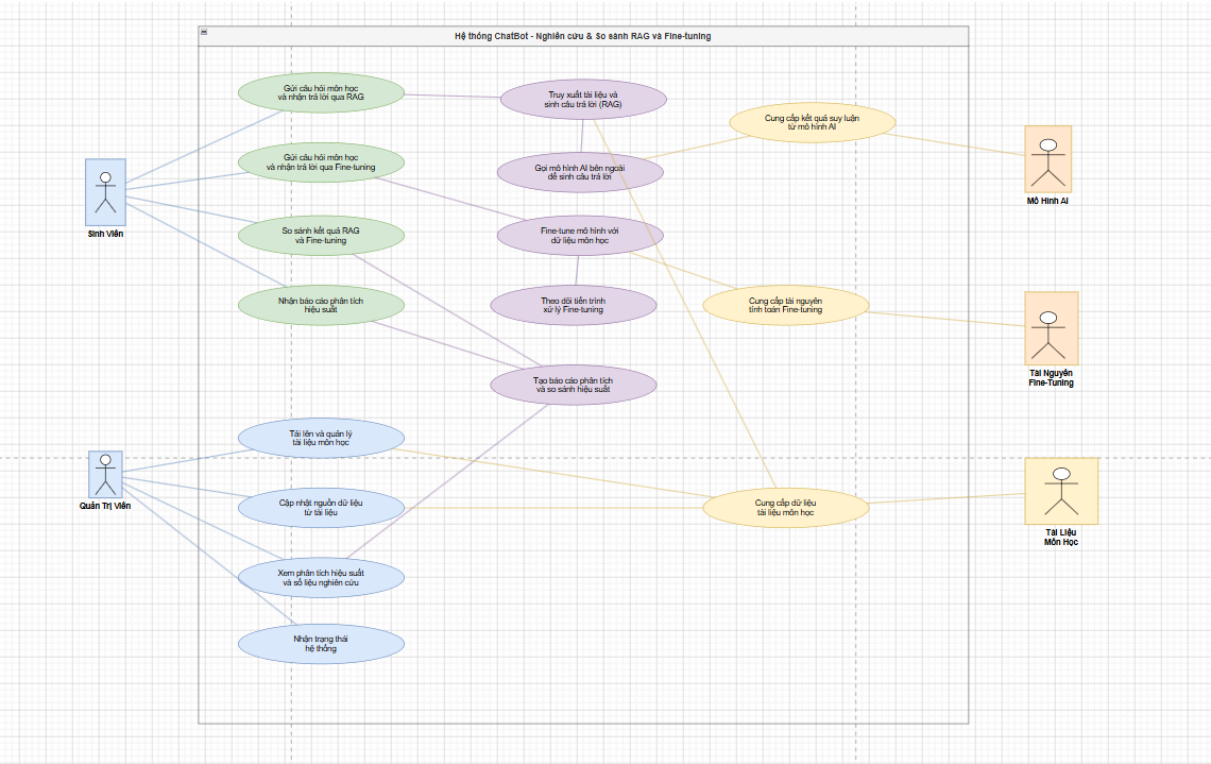


Figure 3.2: Biểu đồ Use Case tổng quan của hệ thống

Biểu đồ Use Case cho thấy hệ thống phân tách rõ rệt quyền hạn giữa hai nhóm người dùng:

Đối với Sinh viên:

- UC-01: Đặt câu hỏi về tài liệu môn học (Chat RAG).
- UC-02: Xem lịch sử hội thoại và tiếp tục ngữ cảnh.
- UC-03: Cấu hình thông số RAG trong Chat Sandbox (Top-K, Embedding, Chunking).
- UC-04: Xem nguồn tài liệu trích dẫn của câu trả lời.

Đối với Quản trị viên:

- UC-05: Tải lên tài liệu học tập (PDF/DOCX).
- UC-06: Quản lý kho tài liệu (xem, xóa, cập nhật metadata).
- UC-07: Theo dõi trạng thái xử lý (Ingestion) của tài liệu.
- UC-08: Chạy kịch bản Benchmark với nhiều cấu hình RAG.
- UC-09: Xem Dashboard tổng quan hiệu năng (biểu đồ Radar, bảng xếp hạng).
- UC-10: Quản lý danh sách người dùng.

3.4 Yêu cầu Chức năng (Functional Requirements)

3.4.1 FR-01: Chức năng Chat RAG (Hỏi đáp bằng AI)

FR-01 – Chat RAG: Hỏi đáp bằng AI

Trigger: Sinh viên nhập câu hỏi vào ô chat và nhấn nút “Gửi” (hoặc Enter).

Actor: Sinh viên

Mục đích: Trả lời câu hỏi của sinh viên dựa trên nội dung tài liệu môn học.

Luồng xử lý chính:

1. Sinh viên nhập câu hỏi vào giao diện Chat Sandbox.
2. Frontend gửi câu hỏi + cấu hình RAG (Top-K, Embedding Model, LLM) qua REST API.
3. Backend mã hóa câu hỏi thành Vector Embedding bằng mô hình được chọn.
4. Backend truy vấn ChromaDB để lấy Top-K đoạn văn bản có độ tương đồng cao nhất.
5. Backend xây dựng Prompt Template với ngữ cảnh từ Top-K đoạn tài liệu.
6. Backend gọi LLM API (Gemini 2.5 Flash) để sinh câu trả lời.

7. Backend trả về câu trả lời kèm danh sách nguồn trích dẫn (tên file, số trang).
8. Frontend hiển thị câu trả lời và lưu vào lịch sử hội thoại.

Luồng thay thế: Nếu không có tài liệu trong ChromaDB, Chatbot thông báo “Không tìm thấy thông tin liên quan trong kho tài liệu.”

Business Rules – FR-01

- **BR-01-01:** Câu trả lời PHẢI dựa trên tài liệu đã được tải lên. LLM không được phép tự tạo thông tin ngoài phạm vi tài liệu.
- **BR-01-02:** Mỗi câu trả lời phải kèm theo ít nhất 1 nguồn trích dẫn (tên file, trang).
- **BR-01-03:** Nếu câu hỏi không liên quan đến tài liệu, hệ thống PHẢI thông báo rõ ràng thay vì trả lời bịa đặt.
- **BR-01-04:** Thời gian phản hồi phải dưới 5 giây trong điều kiện bình thường.
- **BR-01-05:** Lịch sử hội thoại được truyền lại trong context để duy trì ngữ cảnh đa lượt.

3.4.2 FR-02: Chức năng Tải lên Tài liệu (Document Upload & Ingestion)

FR-02 – Document Upload & Ingestion

Trigger: Quản trị viên chọn file và nhấn nút “Upload”.

Actor: Quản trị viên

Mục đích: Xử lý và đưa tài liệu mới vào kho tri thức của hệ thống.

Luồng xử lý chính:

1. Admin tải lên file PDF hoặc DOCX qua giao diện quản lý.
2. Backend xác thực định dạng file và kích thước (tối đa 50MB).
3. Backend khởi động Ingestion Pipeline bất đồng bộ:
 - Trích xuất văn bản thô từ file bằng PyPDF/pdfplumber.
 - Làm sạch văn bản (loại bỏ ký tự nhiễu, chuẩn hóa).
 - Phân mảnh văn bản theo chiến lược Chunking được chọn (Recursive/Fixed-size).
 - Tạo Vector Embedding cho từng chunk bằng mô hình Embedding được chọn.
 - Lưu trữ Vector + metadata vào ChromaDB.

- Cập nhật metadata tài liệu vào MySQL (tên, trang, trạng thái).
4. Trạng thái tài liệu được cập nhật: Đang xử lý → Sẵn sàng.
 5. Frontend hiển thị thông báo thành công và cập nhật danh sách tài liệu.

Business Rules – FR-02

- **BR-02-01:** Chỉ chấp nhận file định dạng PDF và DOCX.
- **BR-02-02:** Kích thước file tối đa 50MB.
- **BR-02-03:** Ingestion Pipeline phải chạy bất đồng bộ, không block giao diện.
- **BR-02-04:** Chunk size mặc định: 1000 ký tự, overlap: 200 ký tự.
- **BR-02-05:** Tài liệu bị lỗi xử lý phải được đánh dấu trạng thái “Lỗi” với thông báo chi tiết.

3.4.3 FR-03: Chức năng Quản lý Tài liệu (Document Management)

FR-03 – Document Management

Trigger: Quản trị viên truy cập trang Quản lý Tài liệu.

Actor: Quản trị viên

Mục đích: Cho phép quản trị viên xem, quản lý và xóa tài liệu trong kho tri thức.

Luồng xử lý chính:

1. Admin xem danh sách toàn bộ tài liệu với metadata: tên file, ngày upload, trạng thái, số chunk.
2. Admin có thể tìm kiếm tài liệu theo tên.
3. Admin có thể xem chi tiết thông tin của một tài liệu.
4. Admin có thể xóa tài liệu: hệ thống xóa cả dữ liệu trong MySQL và ChromaDB.

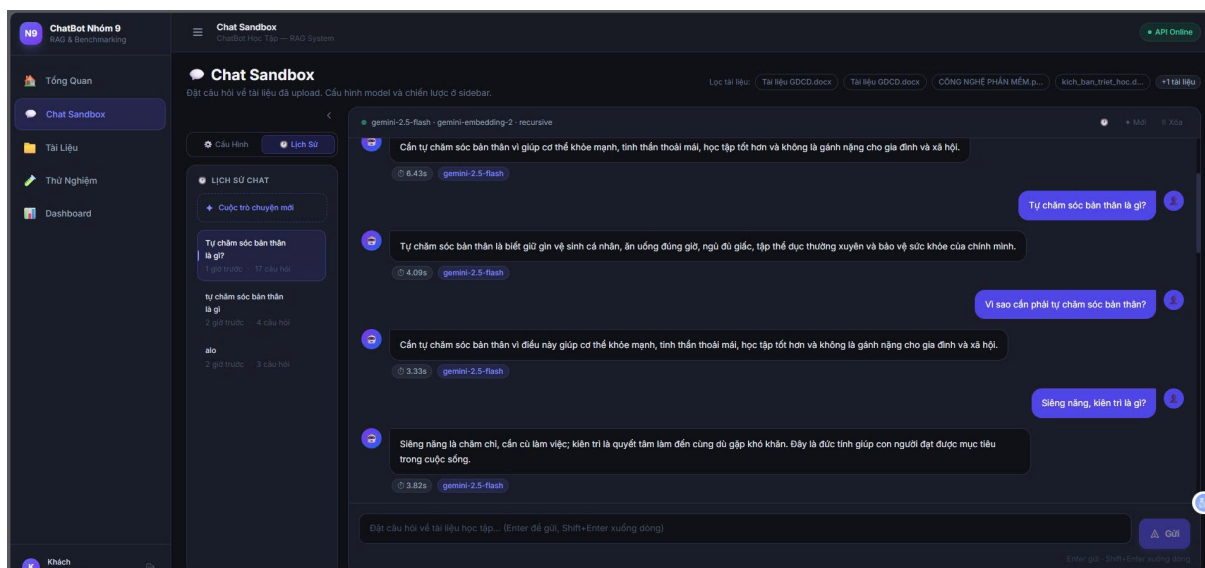


Figure 3.3: Giao diện Quản lý Tài liệu (Document Management)

3.4.4 FR-04: Chức năng Chat Sandbox (Cấu hình RAG)

FR-04 – Chat Sandbox

Trigger: Người dùng truy cập trang Chat Sandbox và điều chỉnh cấu hình.

Actor: Sinh viên, Quản trị viên

Mục đích: Cho phép người dùng thử nghiệm các cấu hình RAG khác nhau trong thời gian thực.

Cấu hình có thể điều chỉnh:

- **Top-K:** Số lượng đoạn tài liệu tối đa được truy xuất (mặc định: 5).
- **Embedding Model:** Google Gemini Embedding hoặc BAAI/bge-m3.
- **LLM Model:** Gemini 2.5 Flash (và các model khác nếu được cấu hình).
- **Chunking Strategy:** Recursive Character hoặc Fixed-size.
- **Tài liệu nguồn:** Chọn một hoặc nhiều tài liệu làm ngữ cảnh.

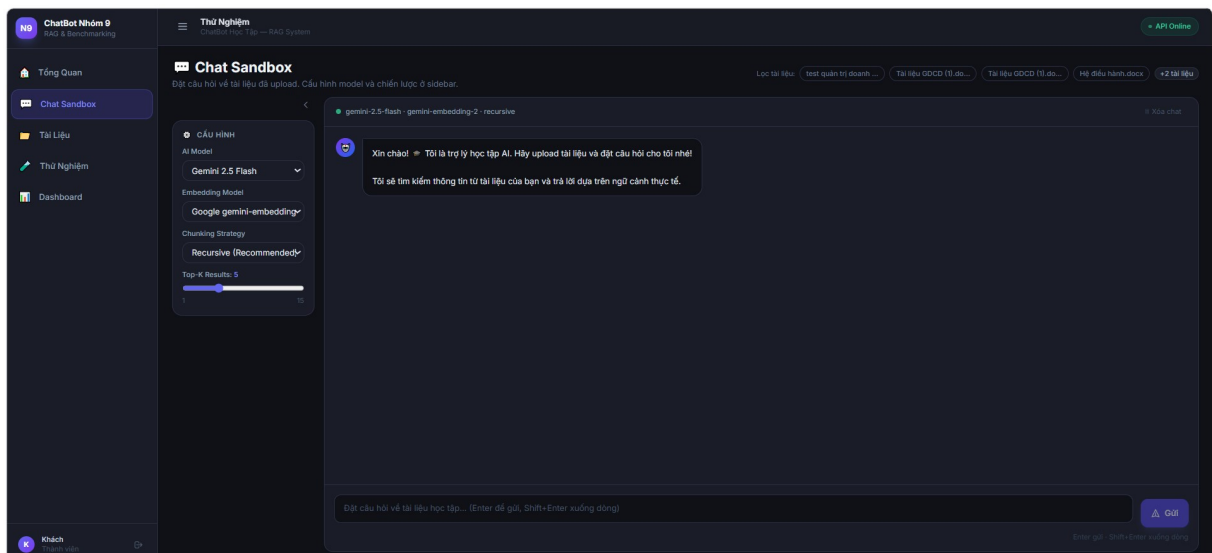


Figure 3.4: Giao diện Chat Sandbox với bảng điều khiển cấu hình RAG

3.4.5 FR-05: Chức năng Benchmark

FR-05 – Benchmark

Trigger: Quản trị viên cấu hình kịch bản và nhấn “Chạy Benchmark”.

Actor: Quản trị viên

Mục đích: Đánh giá định lượng chất lượng các cấu hình RAG khác nhau bằng bộ tiêu chí RAGAS.

Luồng xử lý:

1. Admin cấu hình kịch bản Benchmark: chọn cặp (Chunking Strategy, Embedding Model).
2. Hệ thống nạp bộ câu hỏi kiểm thử (Test Dataset – 50 câu hỏi).
3. Với mỗi câu hỏi, hệ thống chạy toàn bộ chu trình RAG và ghi lại kết quả.
4. Tính toán các chỉ số RAGAS: Accuracy, Relevancy, Faithfulness, Latency.
5. Lưu kết quả vào MySQL và hiển thị lên Dashboard.

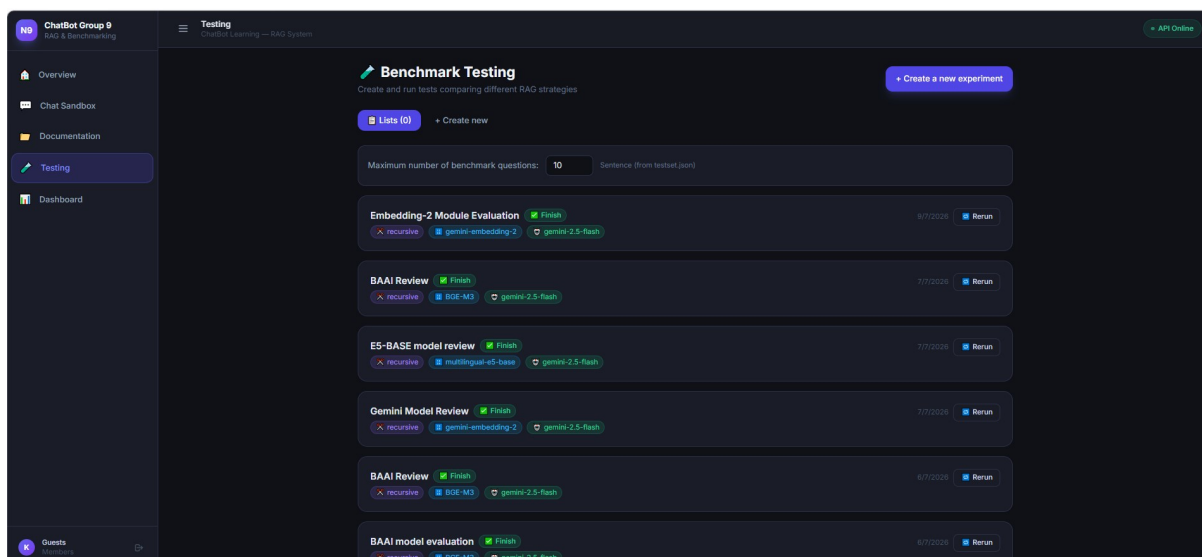


Figure 3.5: Giao diện cấu hình và chạy Benchmark

3.4.6 FR-06: Chức năng Dashboard Phân tích

FR-06 – Dashboard Phân tích

Trigger: Người dùng truy cập trang Dashboard.

Actor: Quản trị viên

Mục đích: Tổng hợp và trực quan hóa kết quả Benchmark qua biểu đồ Radar và bảng xếp hạng.

Nội dung hiển thị:

- Biểu đồ Radar so sánh các cấu hình RAG theo 4 chỉ số RAGAS.
- Bảng xếp hạng các cấu hình từ tốt đến kém.
- Biểu đồ xu hướng Latency theo từng kịch bản Benchmark.
- Tổng số tài liệu, tổng số câu hỏi đã hỏi, thời gian phản hồi trung bình.



Figure 3.6: Dashboard tổng quan các chỉ số RAGAS

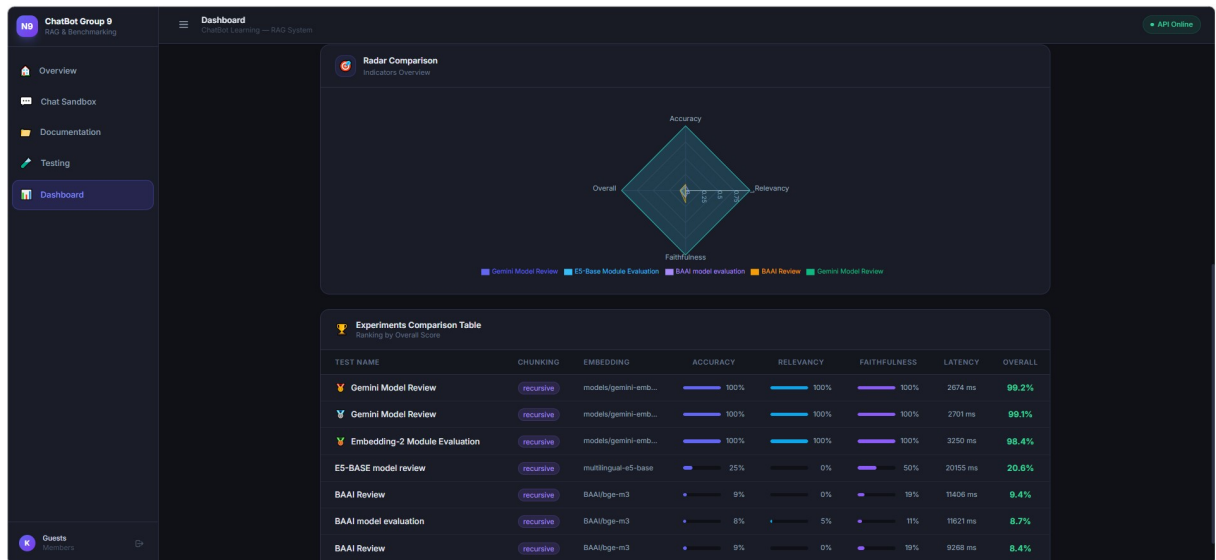


Figure 3.7: Biểu đồ Radar và bảng xếp hạng các cấu hình RAG

3.5 Yêu cầu Phi Chức năng (Non-Functional Requirements)

3.5.1 Hiệu năng (Performance)

- **NFR-01:** Thời gian phản hồi (Latency) cho một câu hỏi thông thường < 5 giây.
- **NFR-02:** Thời gian xử lý Ingestion cho tài liệu 50 trang < 2 phút.
- **NFR-03:** Tốc độ truy vấn ChromaDB < 500ms cho tập dữ liệu $\leq 100,000$ vectors.
- **NFR-04:** Hệ thống phải hỗ trợ ít nhất 10 người dùng đồng thời mà không suy giảm hiệu năng.

3.5.2 Độ chính xác (Accuracy & Faithfulness)

- **NFR-05:** Chỉ số Faithfulness (đo bằng RAGAS) phải đạt ≥ 0.85 .

- **NFR-06:** Chỉ số Relevancy phải đạt ≥ 0.80 .
- **NFR-07:** Hệ thống phải từ chối trả lời (thay vì hallucinate) khi không tìm thấy thông tin liên quan trong tài liệu.

3.5.3 Bảo mật (Security)

- **NFR-08:** Phân quyền rõ ràng: chỉ Quản trị viên được phép xóa tài liệu và chạy Benchmark.
- **NFR-09:** API Key của LLM (Gemini) phải được lưu trong biến môi trường (.env), không hardcode trong mã nguồn.
- **NFR-10:** Dữ liệu người dùng phải được bảo vệ, không lưu thông tin nhạy cảm dạng plain text.

3.5.4 Khả năng Mở rộng (Scalability)

- **NFR-11:** Kiến trúc phải cho phép dễ dàng thay thế/bổ sung mô hình LLM hoặc Embedding Model mà không cần sửa đổi logic nghiệp vụ cốt lõi (Interface Pattern).
- **NFR-12:** ChromaDB phải hỗ trợ tối thiểu 1,000 tài liệu (khoảng 10 triệu chunks) trong tương lai.

3.5.5 Khả dụng (Availability)

- **NFR-13:** Backend FastAPI phải hỗ trợ xử lý bất đồng bộ (Async I/O) để phục vụ nhiều sinh viên cùng lúc.
- **NFR-14:** Hệ thống phải có cơ chế xử lý lỗi (Error Handling) và trả về thông báo lỗi rõ ràng khi API bên ngoài không phản hồi.

3.5.6 Khả năng Bảo trì (Maintainability)

- **NFR-15:** Code phải tuân thủ chuẩn PEP 8 (Python) và ESLint Airbnb (JavaScript).
- **NFR-16:** Tài liệu API phải được sinh tự động bằng Swagger/OpenAPI.
- **NFR-17:** Module hóa rõ ràng: mỗi module (RAG, Benchmark, Auth) là một package độc lập.

3.6 Yêu cầu Giao diện Ngoài (External Interface Requirements)

3.6.1 Giao diện Người dùng (User Interface)

- Giao diện Web hiển thị tốt trên các trình duyệt hiện đại (Chrome, Firefox, Edge).

- Thiết kế Responsive, hỗ trợ màn hình từ 1024px trở lên.
- Hỗ trợ Dark/Light mode (tùy chọn).
- Khu vực Chat phải có thanh cuộn (scroll) khi lịch sử dài.

3.6.2 Giao diện Phần mềm (Software Interface)

Table 3.4: Danh sách giao diện phần mềm bên ngoài

Hệ thống	Giao thức	Mô tả
Google Gemini API	HTTPS/REST	Gọi LLM để sinh câu trả lời và tạo Text Embedding.
BAAI/bge-m3	Python Library	Embedding cục bộ không cần API key.
ChromaDB	Python SDK	Lưu trữ và truy vấn Vector Embedding nội bộ.
MySQL	SQLAlchemy ORM	Lưu trữ metadata, lịch sử chat, kết quả Benchmark.
PyPDF / pdfplumber	Python Library	Trích xuất văn bản từ file PDF.

3.7 Phụ lục Yêu cầu (Requirements Appendix)

3.7.1 Danh sách Thông báo Hệ thống (Application Messages List)

Table 3.5: Danh sách thông báo hệ thống

Mã	Loại	Nội dung thông báo	Trigger
MSG-01	Thành công	“Tài liệu đã được xử lý thành công và sẵn sàng sử dụng.”	Ingestion hoàn tất.
MSG-02	Lỗi	“Định dạng file không được hỗ trợ. Chỉ chấp nhận PDF và DOCX.”	Sai định dạng.
MSG-03	Lỗi	“Không tìm thấy thông tin liên quan trong kho tài liệu.”	ChromaDB trả về rỗng.
MSG-04	Cảnh báo	“Kết nối tới LLM API bị gián đoạn. Vui lòng thử lại.”	API timeout.
MSG-05	Thông tin	“Benchmark đang chạy. Vui lòng chờ...”	Benchmark bắt đầu.
MSG-06	Thành công	“Benchmark hoàn tất. Xem kết quả tại Dashboard.”	Benchmark xong.
MSG-07	Lỗi	“Kích thước file vượt quá giới hạn cho phép (50MB).”	File quá lớn.

3.7.2 Thuật ngữ Giải thích (Glossary)

Table 3.6: Bảng thuật ngữ sử dụng trong tài liệu

Thuật ngữ	Định nghĩa
RAG	Retrieval-Augmented Generation – Kỹ thuật kết hợp truy xuất tài liệu với sinh ngôn ngữ.
LLM	Large Language Model – Mô hình ngôn ngữ lớn (ví dụ: Gemini, GPT).
Embedding	Biểu diễn văn bản dưới dạng vector đa chiều.
Chunking	Kỹ thuật phân mảnh văn bản thành các đoạn nhỏ hơn.
Hallucination	Hiện tượng AI tự bịa đặt thông tin không có trong dữ liệu huấn luyện.
RAGAS	Retrieval Augmented Generation Assessment – Bộ tiêu chí đánh giá hệ thống RAG.
Top-K	Số lượng đoạn tài liệu liên quan nhất được truy xuất từ ChromaDB.
Ingestion Pipeline	Luồng xử lý tự động từ lúc tải file đến khi lưu vào ChromaDB.
Cosine Similarity	Phép đo độ tương đồng giữa hai vector bằng cách đo góc giữa chúng.

3.8 Kết luận Chương

Chương này đã trình bày đầy đủ đặc tả yêu cầu phần mềm (SWR) của hệ thống Chatbot RAG, bao gồm: biểu đồ ngữ cảnh, biểu đồ Use Case tổng quan, đặc tả chi tiết 6 nhóm chức năng chính (với Function Trigger, luồng xử lý và Business Rules), các yêu cầu phi chức năng về hiệu năng/bảo mật/khả năng mở rộng, và phụ lục danh sách thông báo hệ thống. Đây là nền tảng kỹ thuật quan trọng để thiết kế kiến trúc chi tiết trong chương SWD tiếp theo.

CHAPTER 4

SWD – MÔ TẢ THIẾT KẾ PHẦN MỀM

4.1 Tổng quan Thiết kế

Chương này trình bày Mô tả Thiết kế Phần mềm (Software Design Description – **SWD**) cho hệ thống Chatbot RAG. Tài liệu này mô tả chi tiết kiến trúc hệ thống, thiết kế cơ sở dữ liệu và thiết kế các thành phần phần mềm thông qua các biểu đồ UML bao gồm: Use Case chi tiết, Class Diagram, State Machine Diagram, Flowchart và Activity Diagram.

4.2 Thiết kế Kiến trúc Hệ thống (System Architecture)

4.2.1 Kiến trúc Tổng thể

Hệ thống được xây dựng theo mô hình **Client-Server** kết hợp với **AI Pipeline** chuyên biệt cho RAG:

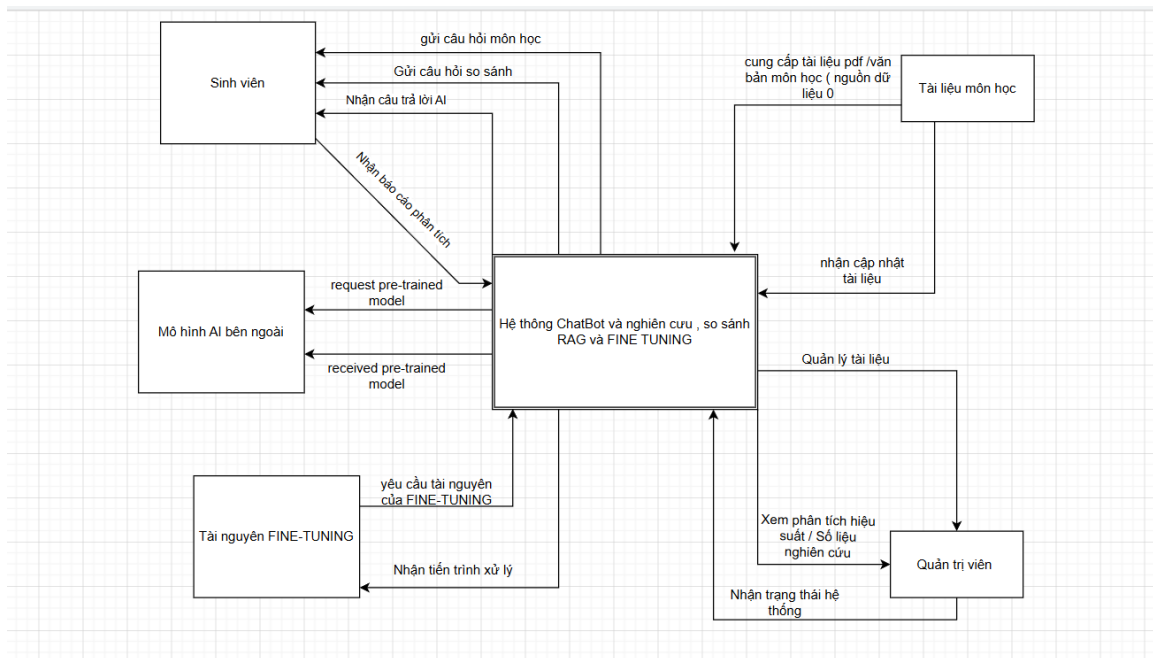


Figure 4.1: Kiến trúc tổng thể hệ thống Chatbot RAG

Kiến trúc hệ thống được phân tách thành 5 lớp rõ ràng:

1. **Presentation Layer (Frontend ReactJS):** Giao diện người dùng, xử lý tương tác và render dữ liệu.

2. **API Gateway Layer (FastAPI Router):** Tiếp nhận, xác thực và định tuyến request.
3. **Business Logic Layer (Services):** Điều phối RAG Pipeline, Benchmark Module, Document Service.
4. **Data Access Layer:** Giao tiếp với ChromaDB (Vector) và MySQL (Quan hệ).
5. **External Services Layer:** Gemini API, BAAI/bge-m3 Embedding.

4.2.2 Package Diagram

4.3 Thiết kế Cơ sở Dữ liệu (Database Design)

4.3.1 Entity Relationship Diagram (ERD)

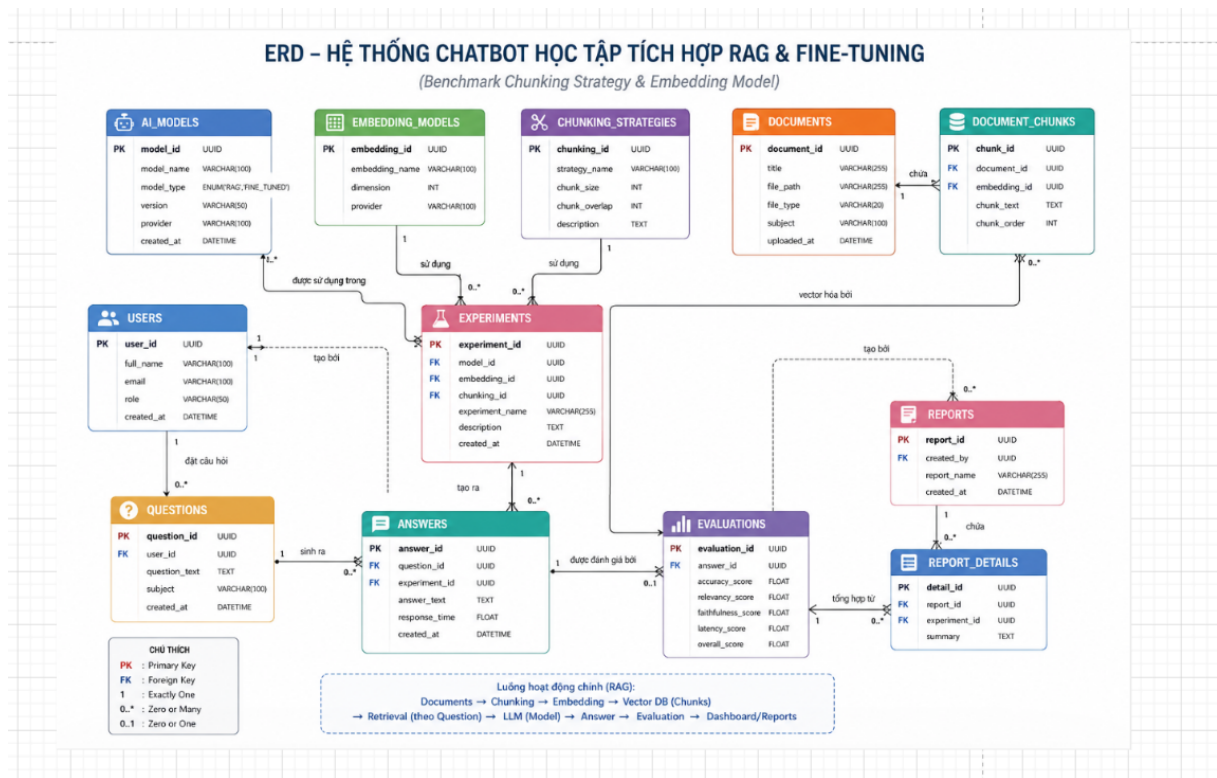


Figure 4.2: Sơ đồ thực thể quan hệ (ERD) của hệ thống

4.3.2 Mô tả Các Bảng Dữ liệu Chính

Table 4.1: Mô tả bảng `users`

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Khóa chính.
username	VARCHAR(100)	NOT NULL, UNIQUE	Tên đăng nhập.
email	VARCHAR(255)	NOT NULL, UNIQUE	Địa chỉ email.
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã băm.
role	ENUM	NOT NULL	Vai trò: student / admin.
created_at	DATETIME	DEFAULT NOW()	Ngày tạo tài khoản.

Table 4.2: Mô tả bảng `documents`

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	PK, AUTO_INCREMENT	Khóa chính.
file_name	VARCHAR(255)	NOT NULL	Tên file gốc.
subject	VARCHAR(100)		Môn học liên quan.
total_chunks	INT	DEFAULT 0	Số lượng chunk sau Ingestion.
status	ENUM	NOT NULL	Trạng thái: processing / ready / error.
uploaded_by	INT	FK → users.id	ID người tải lên.
uploaded_at	DATETIME	DEFAULT NOW()	Thời điểm tải lên.
chroma_collection	VARCHAR(100)		Tên collection trong ChromaDB.

Table 4.3: Mô tả bảng `conversations` và `messages`

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Bảng: conversations			
id	INT	PK	Khóa chính phiên trò chuyện.
user_id	INT	FK → users.id	Người dùng sở hữu phiên.
created_at	DATETIME	DEFAULT NOW()	Thời điểm tạo phiên.
Bảng: messages			
id	INT	PK	Khóa chính tin nhắn.
conversation_id	INT	FK → conversations.id	Phiên trò chuyện.
role	ENUM	NOT NULL	user / assistant.
content	TEXT	NOT NULL	Nội dung tin nhắn.
sources	JSON		Danh sách nguồn trích dẫn.
latency_ms	INT		Thời gian phản hồi (ms).
created_at	DATETIME	DEFAULT NOW()	Thời điểm gửi tin nhắn.

Table 4.4: Mô tả bảng `experiments` và `evaluations`

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Bảng: experiments			
id	INT	PK	Khóa chính kịch bản.
name	VARCHAR(200)	NOT NULL	Tên kịch bản Benchmark.
chunking_strategy	VARCHAR(50)	NOT NULL	Chiến lược Chunking sử dụng.
embedding_model	VARCHAR(100)	NOT NULL	Mô hình Embedding sử dụng.
llm_model	VARCHAR(100)	NOT NULL	Mô hình LLM sử dụng.
avg_accuracy	FLOAT		Điểm Accuracy trung bình.
avg_faithfulness	FLOAT		Điểm Faithfulness trung bình.
avg_relevancy	FLOAT		Điểm Relevancy trung bình.
avg_latency_ms	FLOAT		Latency trung bình (ms).
created_at	DATETIME	DEFAULT NOW()	Thời điểm chạy.
Bảng: evaluations			
id	INT	PK	Khóa chính đánh giá.
experiment_id	INT	FK → experiments.id	Kịch bản tương ứng.
question	TEXT	NOT NULL	Câu hỏi trong test set.
ground_truth	TEXT	NOT NULL	Đáp án chuẩn.
generated_answer	TEXT	NOT NULL	Câu trả lời AI sinh ra.
accuracy	FLOAT		Điểm Accuracy từng câu.
faithfulness	FLOAT		Điểm Faithfulness từng câu.
relevancy	FLOAT		Điểm Relevancy từng câu.
latency_ms	INT		Thời gian xử lý (ms).

4.4 Thiết kế Chi tiết (Detailed Design)

4.4.1 Biểu đồ Lớp (Class Diagram)

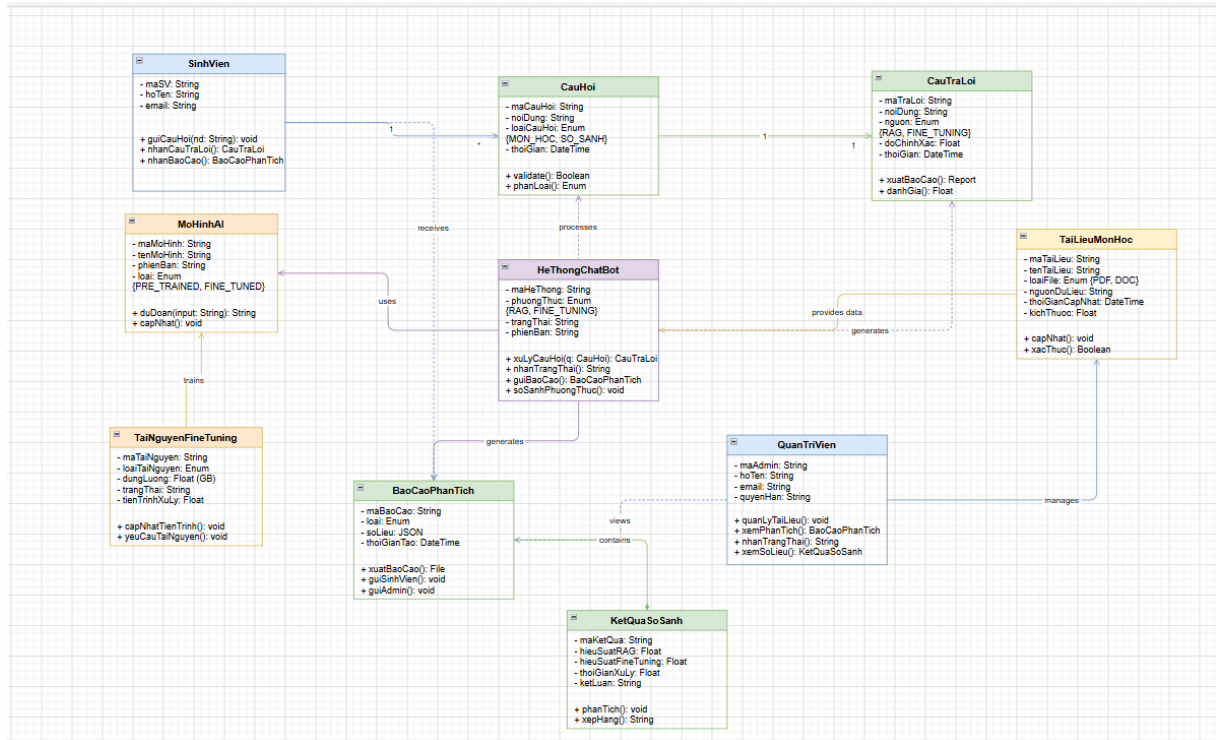


Figure 4.3: Biểu đồ Lớp (Class Diagram) của hệ thống

Hệ thống được thiết kế xung quanh các lớp chính:

Table 4.5: Mô tả các lớp trong Class Diagram

Tên Lớp	Trách nhiệm
User (abstract)	Lớp trừu tượng chung cho người dùng, định nghĩa các thuộc tính cơ bản (id, username, email, role).
SinhVien	Kế thừa User; cung cấp phương thức <code>send_question()</code> , <code>get_history()</code> , <code>configure_rag()</code> .
QuanTriVien	Kế thừa User; bổ sung <code>upload_document()</code> , <code>delete_document()</code> , <code>run_benchmark()</code> .
HeThongChatbot	Lớp điều phối trung tâm (Facade). Nhận <code>CauHoi</code> , điều phối RAG Pipeline, trả về <code>CauTraLoi</code> .
TaiLieuMonHoc	Đóng gói tài liệu: đọc file, chunking, gọi Embedding, lưu ChromaDB.
MoHinhAI (interface)	Interface chung cho LLM và Embedding Model. Phương thức: <code>generate(prompt)</code> , <code>embed(text)</code> .
GeminiAdapter	Triển khai MoHinhAI cho Gemini API.
BGEAdapter	Triển khai MoHinhAI cho BAAI/bge-m3 cục bộ.
ChromaDBRepository	Lớp truy xuất dữ liệu Vector: <code>insert()</code> , <code>search(vector, top_k)</code> , <code>delete()</code> .
BenchmarkService	Điều phối kịch bản Benchmark: nạp test set, vòng lặp đánh giá, tính RAGAS metrics.
BaoCaoPhanTich	Tổng hợp số liệu Benchmark, tạo dữ liệu cho Radar Chart và bảng xếp hạng.

4.4.2 Biểu đồ Trạng thái (State Machine Diagram)

4.4.2.1 State Machine: Vòng đời Tài liệu (Document Lifecycle)

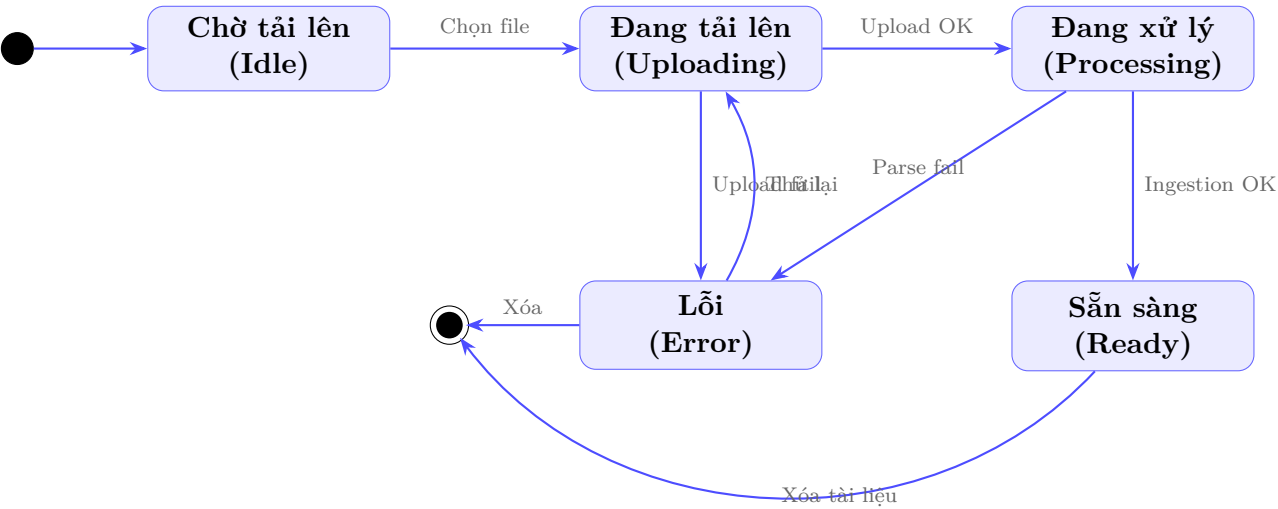


Figure 4.4: State Machine Diagram: Vòng đời Tài liệu (Document Lifecycle)

Mô tả các trạng thái:

Table 4.6: Mô tả trạng thái tài liệu

Trạng thái	Điều kiện vào	Hành động
Idle	Khởi tạo hệ thống	Hiển thị nút Upload.
Uploading	Admin chọn file và nhấn Upload	Gửi file lên server, hiển thị progress bar.
Processing	Upload thành công	Chạy Ingestion Pipeline: extract → chunk → embed → store.
Ready	Ingestion Pipeline hoàn tất	Tài liệu sẵn sàng cho chat RAG.
Error	Upload hoặc Parsing thất bại	Ghi log lỗi, hiển thị thông báo, cho phép thử lại.

4.4.2.2 State Machine: Vòng đời Phiên Hội thoại (Chat Session)

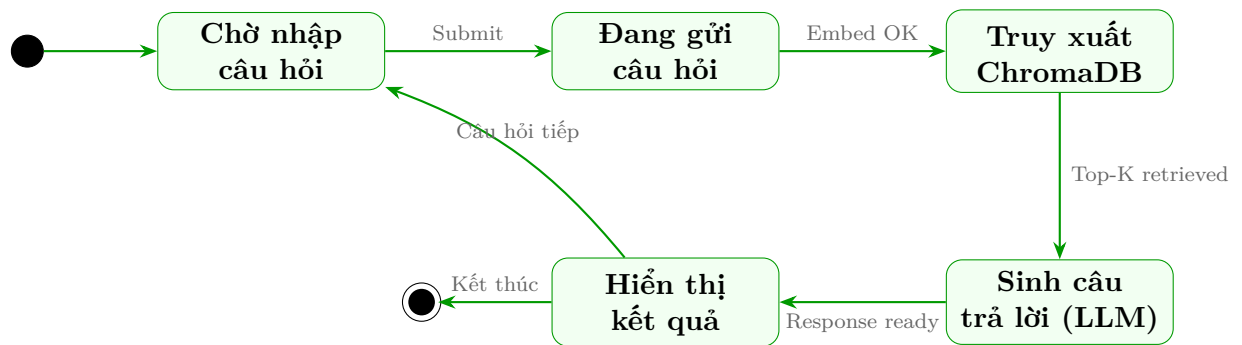


Figure 4.5: State Machine Diagram: Vòng đời Phiên Hội thoại (Chat Session)

4.4.3 Biểu đồ Luồng (Flowchart)

4.4.3.1 Flowchart: RAG Pipeline (Luồng Hỏi đáp)

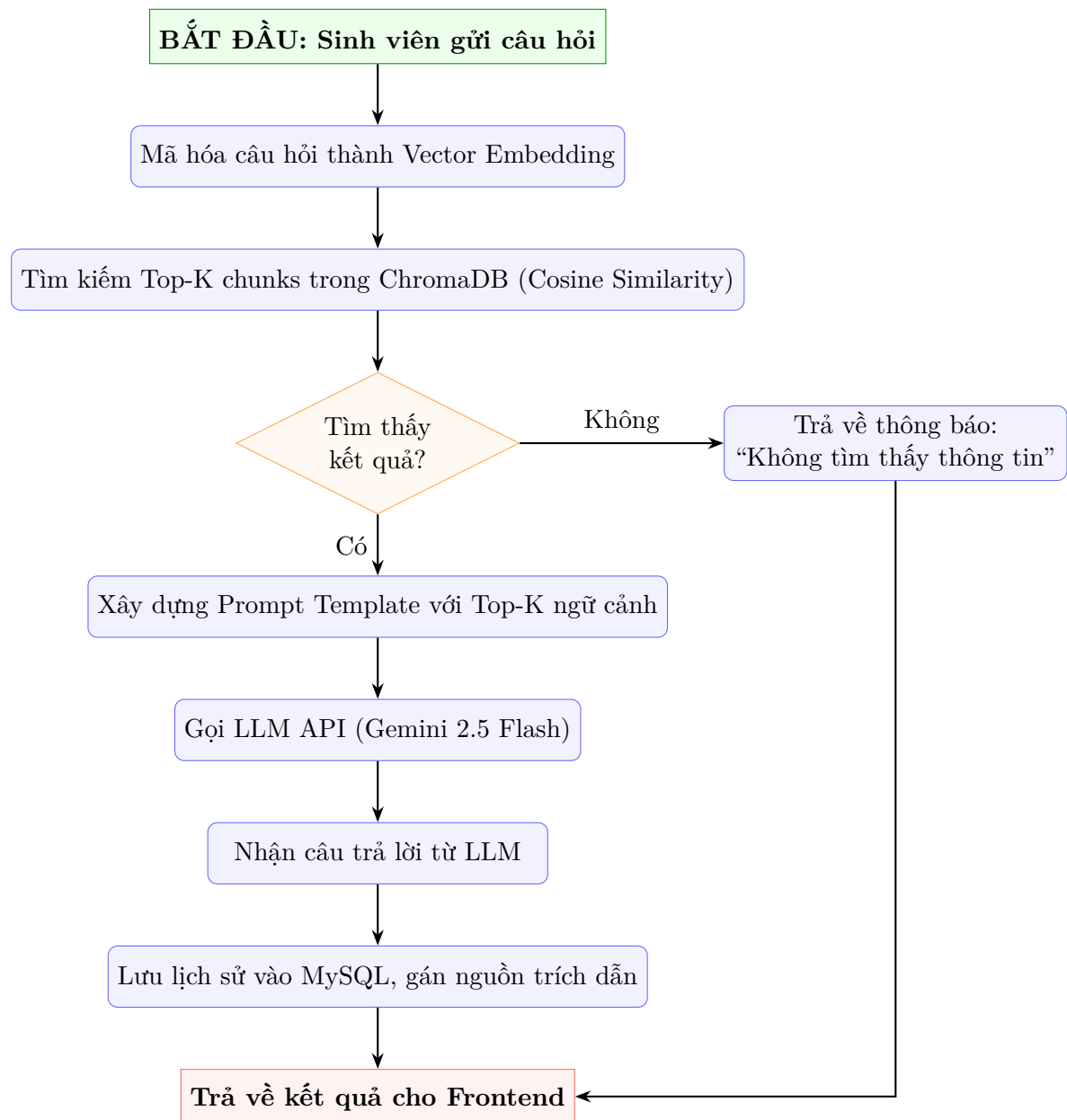


Figure 4.6: Flowchart: RAG Pipeline – Luồng xử lý câu hỏi hỏi đáp

4.4.3.2 Flowchart: Ingestion Pipeline (Luồng Xử lý Tài liệu)

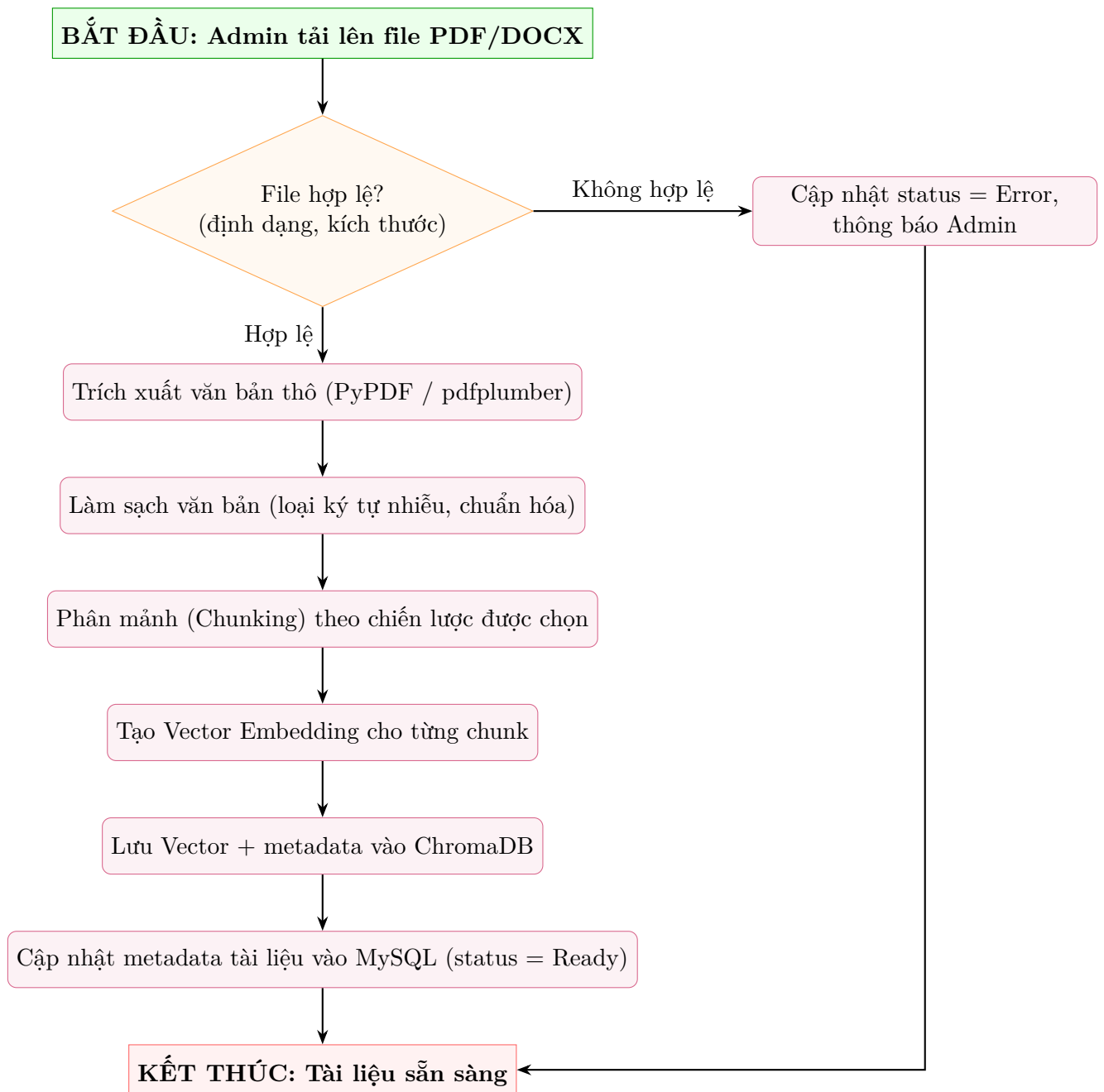


Figure 4.7: Flowchart: Ingestion Pipeline – Luồng xử lý tài liệu

4.4.3.3 Flowchart: Benchmark Pipeline

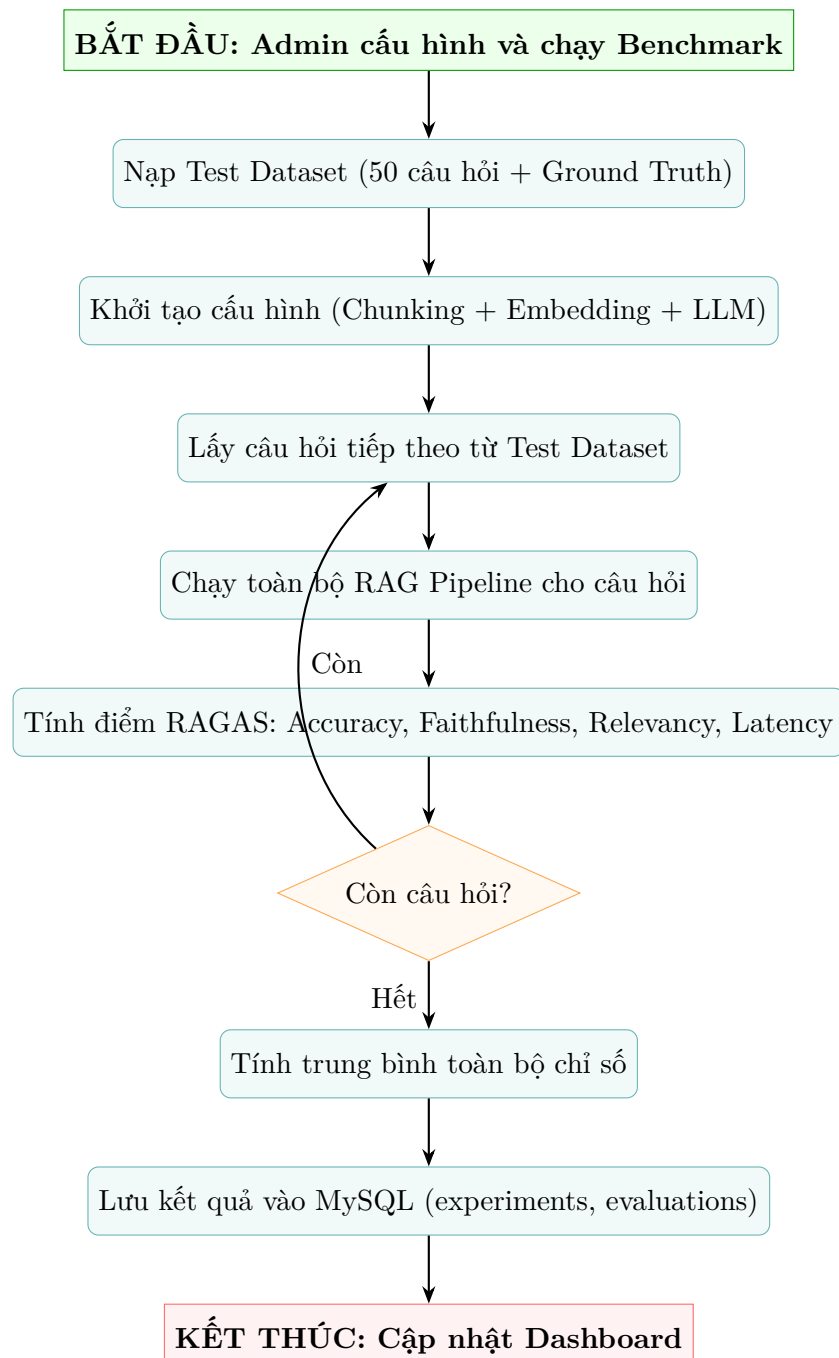


Figure 4.8: Flowchart: Benchmark Pipeline

4.4.4 Biểu đồ Hoạt động (Activity Diagram)

4.4.4.1 Activity Diagram: Tương tác Chat giữa Sinh viên và Hệ thống

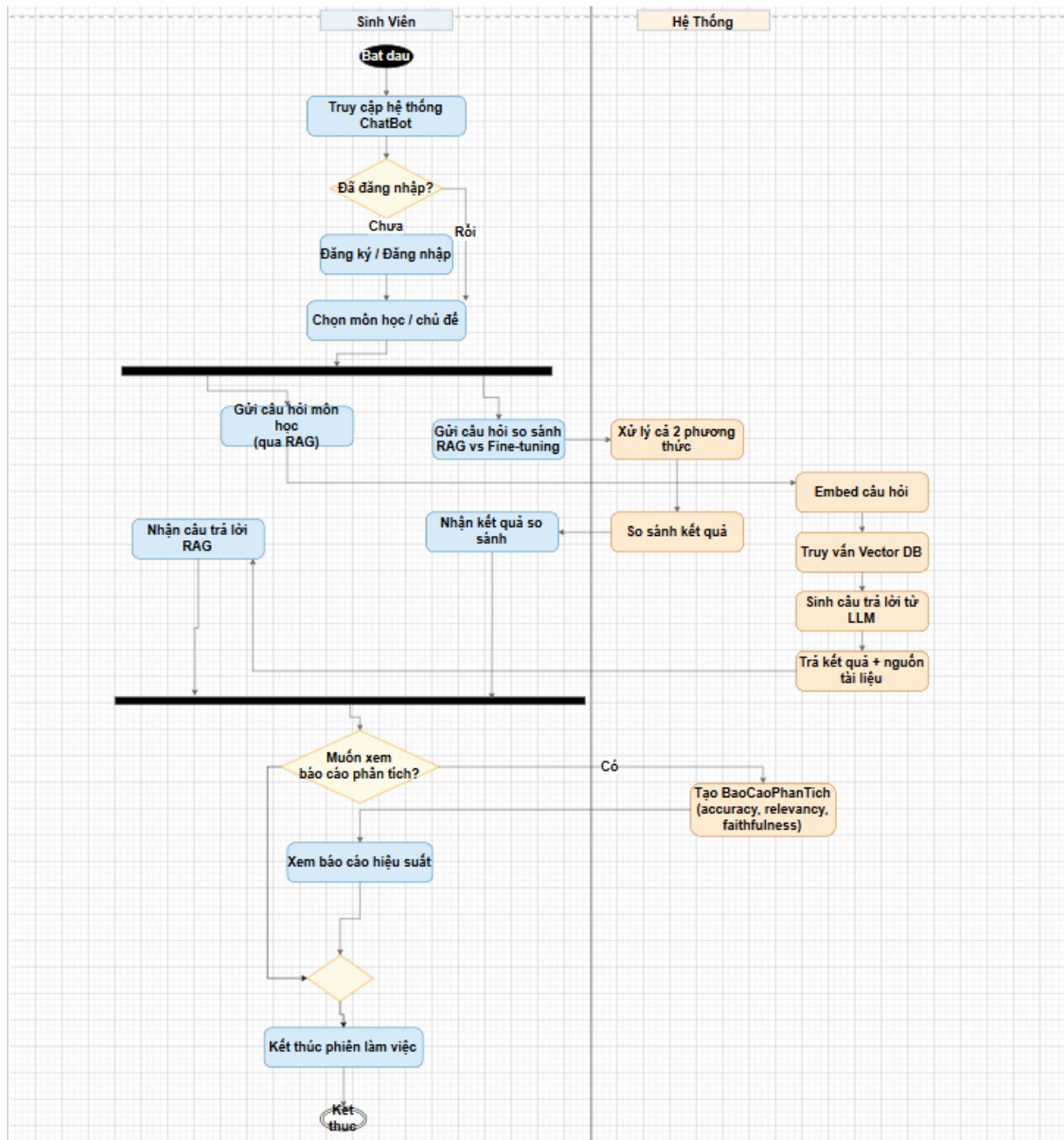


Figure 4.9: Activity Diagram: Tương tác Chat giữa Sinh viên và Hệ thống

4.4.4.2 Activity Diagram: Quản trị viên Upload và Quản lý Tài liệu

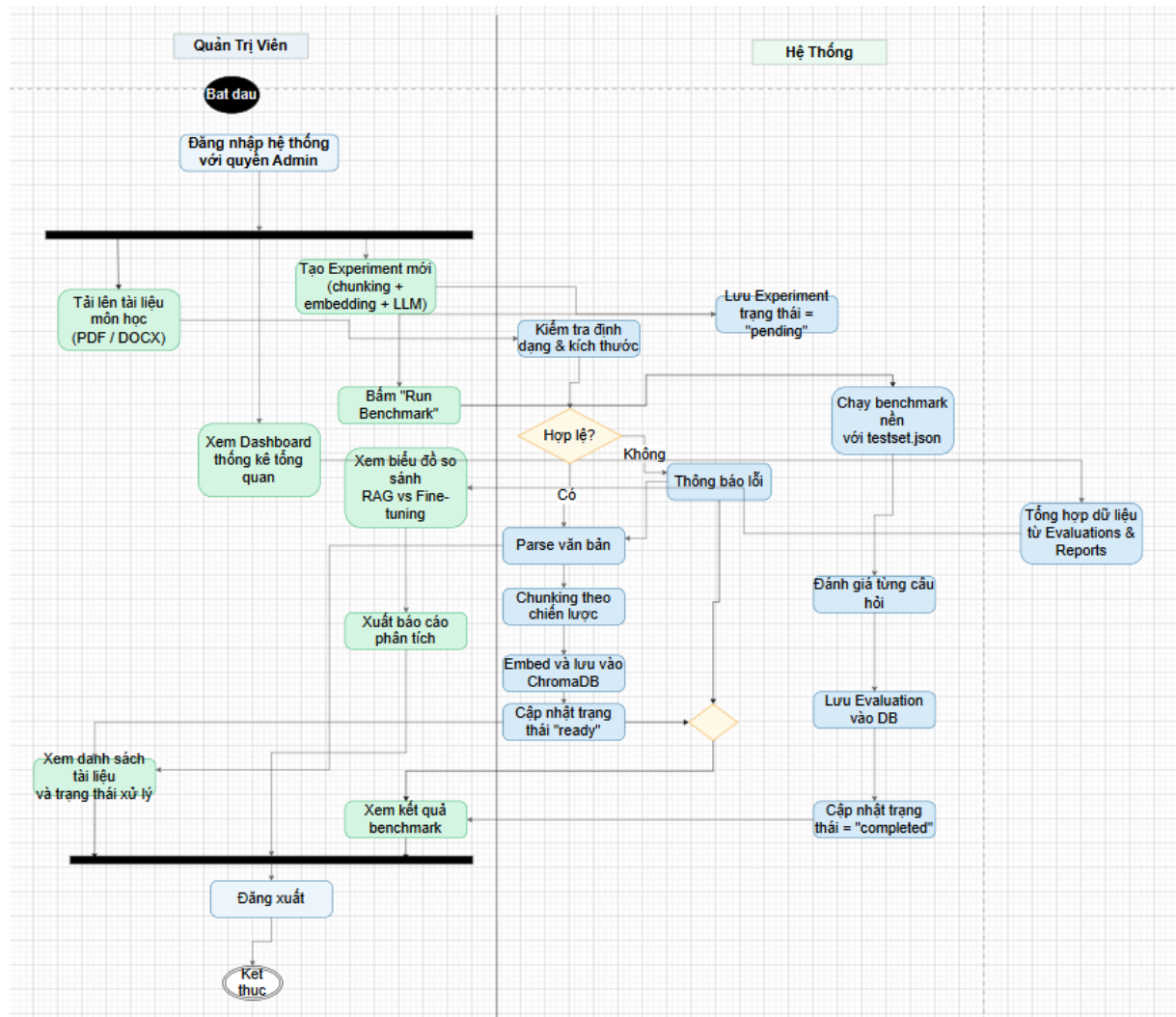


Figure 4.10: Activity Diagram: Quản trị viên Upload và Quản lý Tài liệu

4.4.4.3 Activity Diagram: Quy trình Benchmark

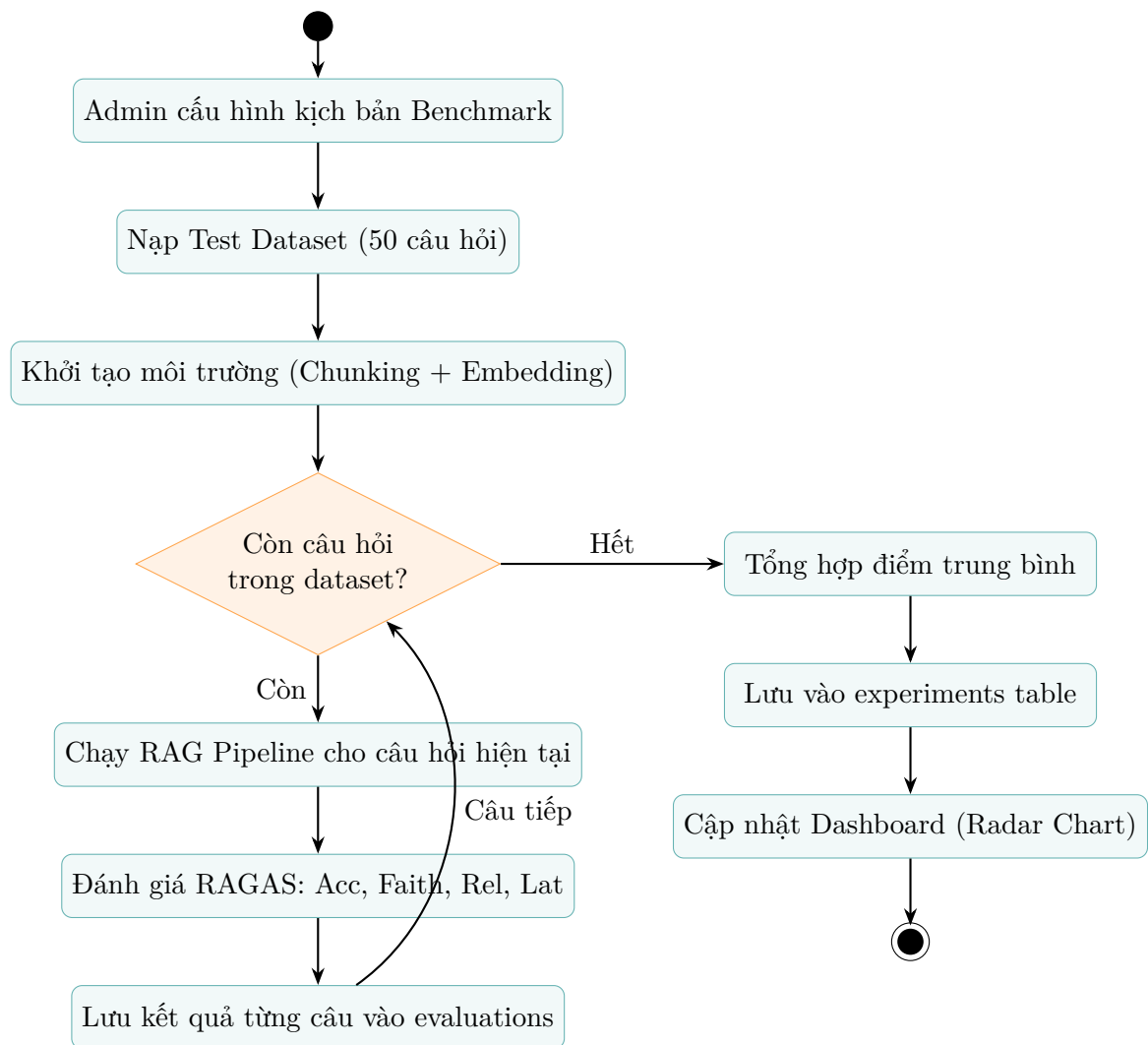


Figure 4.11: Activity Diagram: Quy trình Benchmark đánh giá RAG

4.4.5 Biểu đồ Use Case Chi tiết (Detailed Use Case)

4.4.5.1 UC-01: Đặt câu hỏi Chat RAG

Table 4.7: Đặc tả Use Case UC-01: Chat RAG

Mục	Nội dung
Use Case ID	UC-01
Tên	Chat RAG – Hỏi đáp bằng AI từ tài liệu
Actor	Sinh viên (Primary), LLM (Secondary), ChromaDB (Secondary)
Pre-condition	Hệ thống có ít nhất 1 tài liệu ở trạng thái “Ready” trong ChromaDB.
Trigger	Sinh viên nhập câu hỏi và nhấn “Gửi”.
Main Flow	1. Sinh viên nhập câu hỏi vào Chat Sandbox. 2. Hệ thống nhận request và mã hóa câu hỏi thành Vector. 3. ChromaDB tìm Top-K chunks tương đồng nhất. 4. Backend xây dựng Prompt với ngữ cảnh từ các chunks. 5. Backend gọi Gemini API, nhận câu trả lời. 6. Hệ thống trả về câu trả lời + danh sách nguồn trích dẫn. 7. Frontend hiển thị và lưu lịch sử.
Alternative Flow	Nếu ChromaDB không tìm thấy kết quả → Hệ thống trả về MSG-03.
Post-condition	Câu hỏi và câu trả lời được lưu vào bảng <code>messages</code> .

4.4.5.2 UC-08: Chạy Benchmark

Table 4.8: Đặc tả Use Case UC-08: Benchmark

Mục	Nội dung
Use Case ID	UC-08
Tên	Chạy Benchmark đánh giá cấu hình RAG
Actor	Quản trị viên (Primary)
Pre-condition	Admin đã đăng nhập; Test Dataset tồn tại; ít nhất 1 tài liệu trong ChromaDB.
Trigger	Admin cấu hình kịch bản và nhấn “Chạy Benchmark”.
Main Flow	1. Admin chọn cấu hình: Chunking Strategy + Embedding Model. 2. Hệ thống nạp Test Dataset (50 câu hỏi + Ground Truth). 3. Với mỗi câu hỏi: chạy RAG Pipeline và tính điểm RAGAS. 4. Hệ thống tính điểm trung bình toàn bộ kịch bản. 5. Lưu kết quả vào <code>experiments</code> và <code>evaluations</code> . 6. Cập nhật Dashboard với kết quả mới.
Alternative Flow	Nếu LLM API lỗi → Ghi log lỗi và thông báo Admin.
Post-condition	Kết quả Benchmark được lưu và hiển thị trên Dashboard.

4.5 Thiết kế Giao diện Người dùng (UI Design)

4.5.1 Giao diện Trang chủ

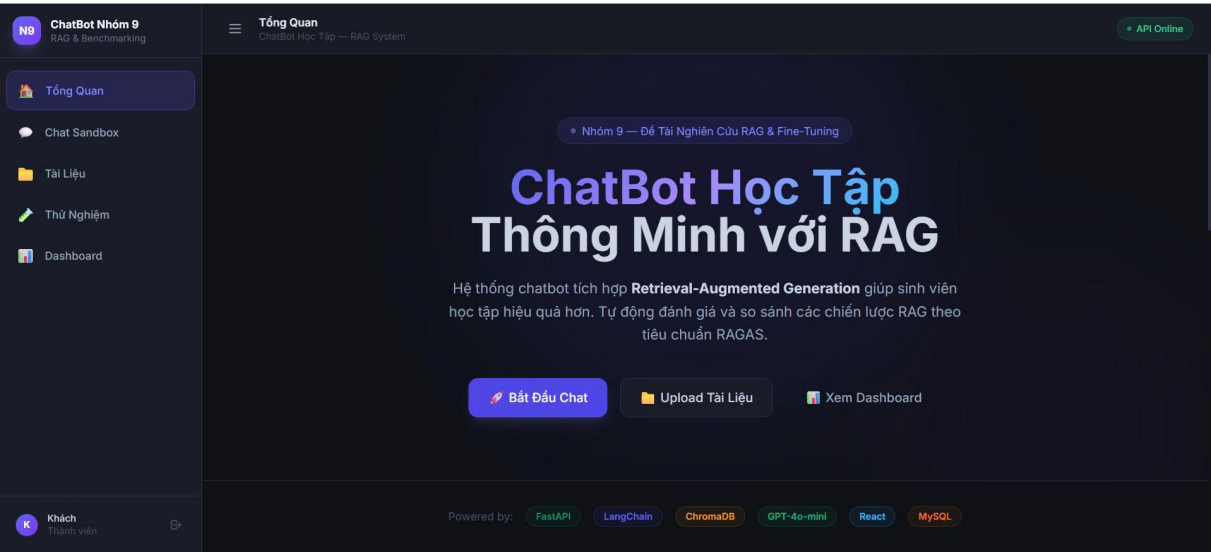


Figure 4.12: Giao diện Trang chủ hệ thống

4.5.2 Giao diện Chat Sandbox

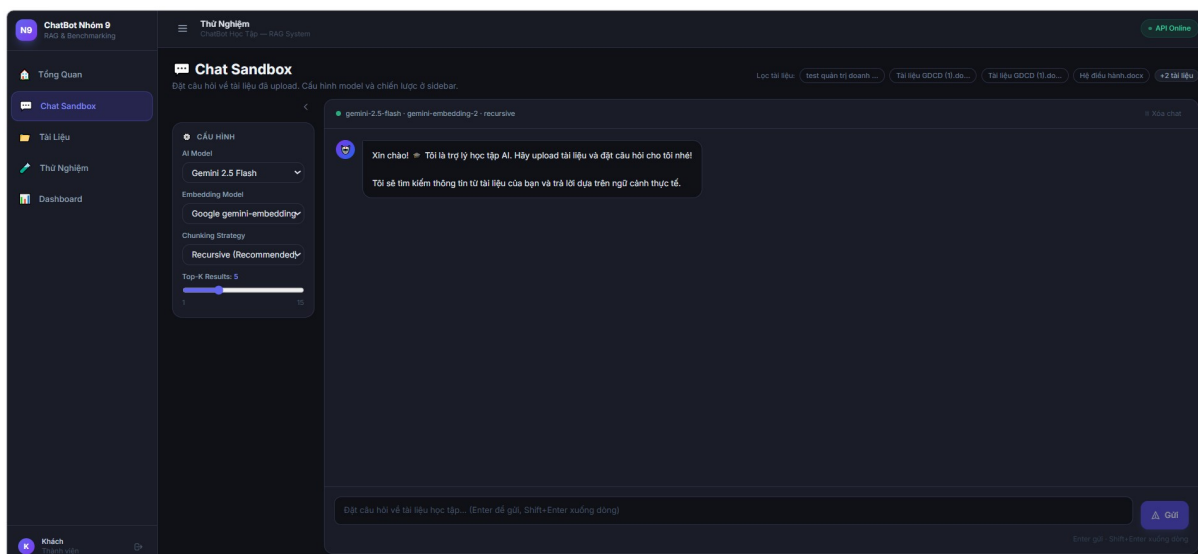


Figure 4.13: Giao diện Chat Sandbox với bảng cấu hình RAG

4.6 Kết luận Chương

Chương này đã trình bày đầy đủ tài liệu thiết kế phần mềm (SWD) bao gồm: kiến trúc hệ thống 5 lớp, Package Diagram, ERD chi tiết với mô tả các bảng, Class Diagram với 11 lớp chính, State Machine Diagram (vòng đời tài liệu và phiên hội thoại), 3 Flowchart (RAG Pipeline, Ingestion Pipeline, Benchmark Pipeline), 3 Activity Diagram (Chat, Upload, Benchmark) và đặc tả Use Case chi tiết. Tất cả thiết kế được xây dựng nhất quán với SWR và tạo nền tảng kỹ thuật vững chắc cho giai đoạn triển khai và kiểm thử.

CHAPTER 5

SWT – TÀI LIỆU KIỂM THỬ PHẦN MỀM

5.1 Tổng quan Tài liệu Kiểm thử

Chương này trình bày **Tài liệu Kiểm thử Phần mềm (Software Testing Documentation – SWT)** cho hệ thống Chatbot RAG. Mục tiêu của kiểm thử không chỉ đảm bảo các chức năng hoạt động đúng theo SWR, mà quan trọng hơn là đánh giá định lượng chất lượng của hệ thống AI thông qua bộ tiêu chuẩn RAGAS.

Table 5.1: Thông tin tài liệu SWT

Mục	Nội dung
Phiên bản	1.0
Ngày kiểm thử	06-07/2026
Người kiểm thử	Trần Thị Kiều Trinh (Lead), hỗ trợ bởi các thành viên nhóm
Công cụ kiểm thử	Pytest, Postman, RAGAS Framework, Selenium (Frontend)

5.2 Phạm vi Kiểm thử (Testing Scope)

5.2.1 Trong phạm vi (In-scope)

- Kiểm thử chức năng (Functional Testing) cho tất cả Use Case trong SWR: UC-01 đến UC-10.
- Kiểm thử API (Backend) bằng Postman cho tất cả endpoint FastAPI.
- Kiểm thử tích hợp (Integration Testing) giữa Backend – ChromaDB – LLM.
- Kiểm thử hiệu năng (Performance Testing) đo Latency của RAG Pipeline.
- Kiểm thử AI (RAGAS Benchmark) đánh giá Accuracy, Faithfulness, Relevancy.
- Kiểm thử phi chức năng: xử lý lỗi, thông báo hệ thống, phân quyền.

5.2.2 Ngoài phạm vi (Out-of-scope)

- Kiểm thử bảo mật chuyên sâu (Penetration Testing).
- Kiểm thử tải (Load Testing) quy mô lớn (> 100 người dùng đồng thời).
- Kiểm thử trên thiết bị di động.

- Kiểm thử khả năng phục hồi sau thảm họa (Disaster Recovery).

5.3 Chiến lược Kiểm thử (Testing Strategy)

5.3.1 Mức độ Kiểm thử (Testing Levels)

Table 5.2: Các mức độ kiểm thử áp dụng

Mức độ	Mô tả	Công cụ
Unit Testing	Kiểm thử từng hàm/module độc lập (RAG Service, Embedding, Chunking).	Pytest
Integration Testing	Kiểm thử luồng dữ liệu giữa các module (Backend → ChromaDB → LLM).	Pytest + Postman
System Testing	Kiểm thử toàn bộ hệ thống theo kịch bản người dùng.	Postman, Manual
AI Quality Testing	Đánh giá chất lượng câu trả lời AI bằng RAGAS Benchmark.	RAGAS Framework
UAT	Người dùng thực tế (sinh viên) dùng thử và phản hồi.	Manual

5.3.2 Loại Kiểm thử (Testing Types)

- **Kiểm thử chức năng (Functional):** Xác nhận từng chức năng hoạt động đúng theo đặc tả SWR.
- **Kiểm thử hiệu năng (Performance):** Đo Latency của RAG Pipeline dưới điều kiện bình thường.
- **Kiểm thử phân quyền (Authorization):** Xác nhận Sinh viên không thể thực hiện tác vụ Admin.
- **Kiểm thử xử lý lỗi (Error Handling):** Xác nhận hệ thống trả về thông báo lỗi đúng chuẩn.
- **Kiểm thử biên (Boundary):** Kiểm thử file upload vượt 50MB, Top-K = 0, câu hỏi rỗng.

5.4 Kế hoạch Kiểm thử (Test Plan)

5.4.1 Bảng Test Cases – Kiểm thử Chức năng

Table 5.3: Test Cases kiểm thử chức năng hệ thống

TC-ID	Tên Test Case	Input	Expected Output	Kết quả
TC-01	Chat RAG: Câu hỏi hợp lệ	Câu hỏi có trong tài liệu	Câu trả lời chính xác + nguồn trích dẫn	Pass
TC-02	Chat RAG: Câu hỏi ngoài phạm vi	Câu hỏi không liên quan tài liệu	Thông báo MSG-03	Pass
TC-03	Chat RAG: Câu hỏi rỗng	Chuỗi rỗng	Thông báo lỗi validation	Pass
TC-04	Chat RAG: Duy trì ngữ cảnh	Câu hỏi liên quan câu trước	Trả lời có ngữ cảnh từ lịch sử	Pass
TC-05	Chat RAG: Thay đổi Top-K	Top-K = 1 / 5 / 10	Số lượng sources trả về đúng	Pass
TC-06	Upload: File PDF hợp lệ	File PDF $\leq$ 50MB	Status = Processing $\rightarrow$ Ready	Pass
TC-07	Upload: File DOCX hợp lệ	File DOCX $\leq$ 50MB	Status = Processing $\rightarrow$ Ready	Pass
TC-08	Upload: File sai định dạng	File .jpg, .xlsx	Thông báo MSG-02	Pass
TC-09	Upload: File quá lớn	File > 50MB	Thông báo MSG-07	Pass
TC-10	Upload: File PDF bị hỏng	File PDF corrupt	Status = Error + thông báo	Pass
TC-11	Xóa tài liệu	Chọn tài liệu đã upload	Tài liệu bị xóa khỏi cả MySQL và ChromaDB	Pass
TC-12	Xem danh sách tài liệu	Không có filter	Hiển thị đúng danh sách + metadata	Pass
TC-13	Phân quyền: Sinh viên xóa tài liệu	Sinh viên gọi DELETE /documents/1	HTTP 403 Forbidden	Pass
TC-14	Phân quyền: Sinh viên chạy Benchmark	Sinh viên gọi POST /benchmark/run	HTTP 403 Forbidden	Pass

TC-ID	Tên Test Case	Input	Expected Output	Kết quả
TC-15	Benchmark: Chạy kịch bản hoàn chỉnh	50 câu hỏi, Recursive + Gemini Embedding	Điểm RAGAS được tính và lưu vào DB	Pass
TC-16	Benchmark: Dashboard cập nhật	Sau khi benchmark xong	Dashboard hiển thị biểu đồ Radar mới	Pass
TC-17	Hiệu năng: Latency câu hỏi thông thường	1 câu hỏi, mạng ổn định	Latency < 5 giây	Pass
TC-18	Hiệu năng: Ingestion tài liệu 50 trang	File PDF 50 trang	Ingestion hoàn tất < 2 phút	Pass
TC-19	Xử lý lỗi: LLM API không phản hồi	Mock Gemini timeout	Thông báo MSG-04, không crash	Pass
TC-20	Xử lý lỗi: ChromaDB không kết nối	Mock ChromaDB down	Thông báo lỗi kết nối DB	Pass

5.5 Môi trường Kiểm thử Benchmark (RAGAS)

5.5.1 Cấu hình Môi trường

Table 5.4: Cấu hình môi trường kiểm thử Benchmark

Thành phần	Cấu hình
Phần cứng	CPU: Intel Core i7-12700H, RAM: 16GB, GPU: NVIDIA RTX 3060
LLM (cố định)	Gemini 2.5 Flash (via Google AI API)
Chunking Strategies	Recursive Character Text Splitter <i>vs</i> Fixed-size Splitter
Chunk Size	1000 ký tự, Overlap: 200 ký tự (đồng nhất cho cả hai)
Embedding Models	Google Gemini Embedding <i>vs</i> BAAI/bge-m3 (local)
Top-K	5 (mặc định cho toàn bộ kịch bản)
Test Dataset	50 câu hỏi + Ground Truth trích từ giáo trình Công nghệ Phần mềm
Framework đánh giá	RAGAS (Python library)

5.5.2 Tập Dữ liệu Kiểm thử (Test Dataset)

Bộ test dataset gồm 50 câu hỏi đa dạng, được phân loại theo độ khó:

Table 5.5: Phân loại câu hỏi trong Test Dataset

Loại câu hỏi	Số lượng	Ví dụ
Nhận biết định nghĩa	15	“RAG là gì?”
Hiểu biết quy trình	15	“Quy trình Ingestion Pipeline gồm những bước nào?”
Suy luận & ứng dụng	12	“Tại sao Recursive Chunking cho kết quả tốt hơn Fixed-size?”
Tổng hợp nhiều nguồn	8	“So sánh ưu nhược điểm của Gemini Embedding và BAAI/bge-m3.”
Tổng	50	

5.6 Kết quả Kiểm thử (Test Reports)

5.6.1 Kết quả RAGAS Benchmark

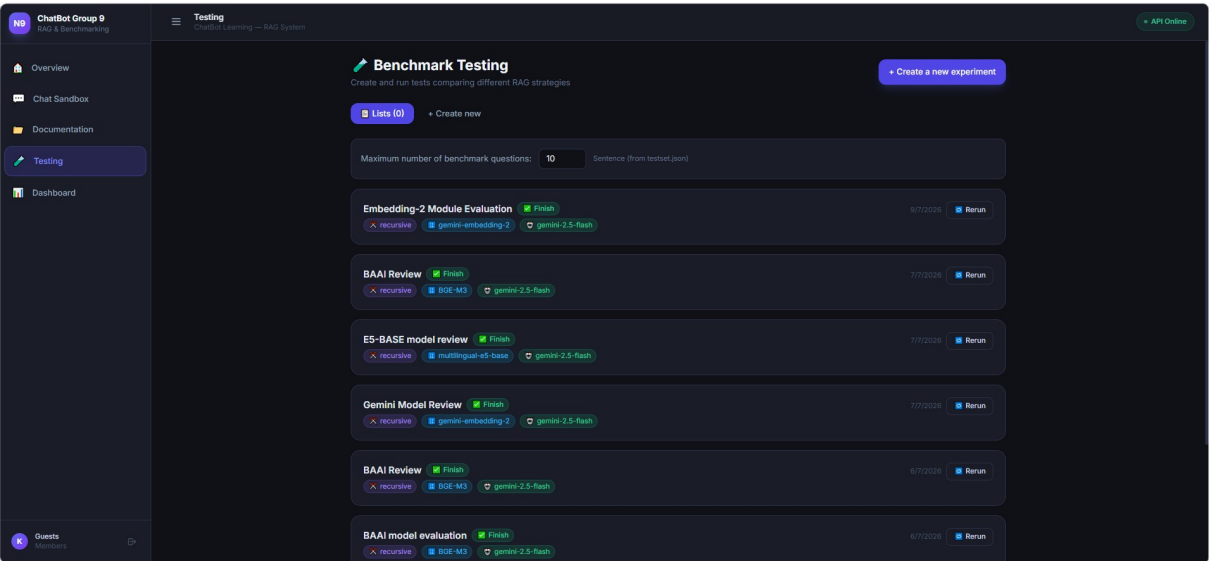


Figure 5.1: Giao diện cấu hình và khởi chạy Benchmark



Figure 5.2: Dashboard tổng quan kết quả RAGAS sau Benchmark

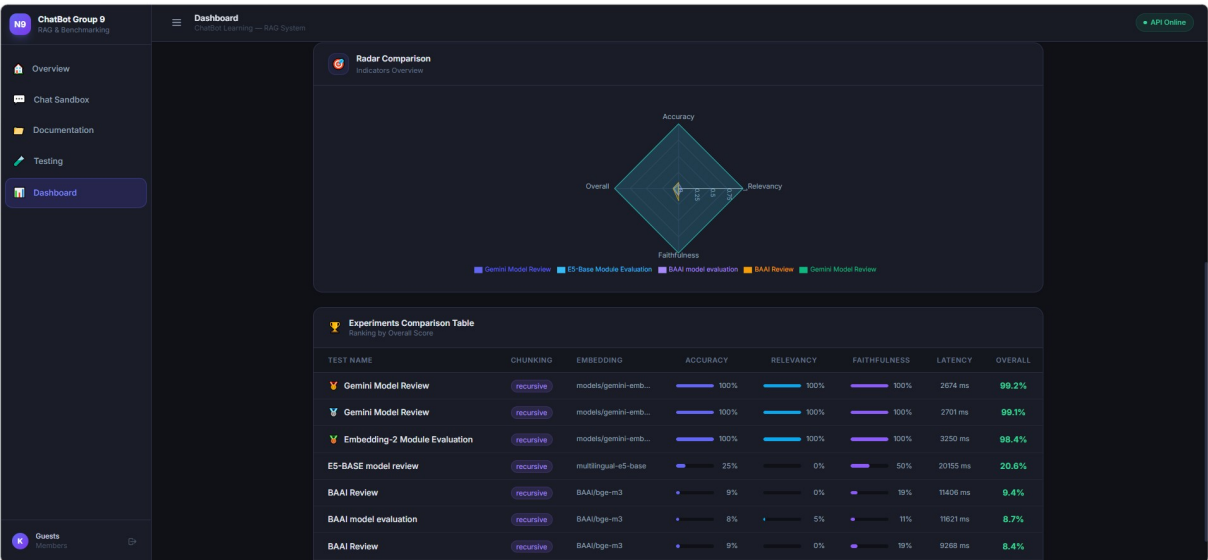


Figure 5.3: Biểu đồ Radar và Bảng xếp hạng các cấu hình RAG

5.6.2 Bảng So sánh Kết quả 4 Cấu hình RAG

Table 5.6: Kết quả Benchmark so sánh 4 cấu hình RAG (LLM cố định: Gemini 2.5 Flash)

Cấu hình	Accuracy	Faithfulness	Relevancy	Latency (s)
Recursive + Gemini Embed	0.89	0.94	0.88	4.1
Recursive + BAAI/bge-m3	0.85	0.92	0.86	3.2
Fixed-size + Gemini Embed	0.81	0.91	0.72	4.3
Fixed-size + BAAI/bge-m3	0.78	0.90	0.70	3.4

5.6.3 Phân tích Kết quả

5.6.3.1 Phân tích Chiến lược Phân mảnh (Chunking)

Kết quả Benchmark cho thấy chiến lược **Recursive Chunking** áp đảo hoàn toàn Fixed-size Chunking về chỉ số Relevancy (đạt 0.88 so với 0.72 của Fixed-size). Nguyên nhân là:

- **Fixed-size Chunking** cắt văn bản khi đạt giới hạn ký tự, bất kể cấu trúc ngữ pháp. Điều này dẫn đến các chunk bị “đứt câu” giữa chừng, làm mất ngữ cảnh quan trọng.
- **Recursive Chunking** ưu tiên cắt tại các điểm tự nhiên (dấu chấm câu, ngắt đoạn, tiêu đề). Các chunk giữ được trọn vẹn ý nghĩa ngữ nghĩa, giúp LLM có nền tảng tốt hơn để tổng hợp câu trả lời.

5.6.3.2 Phân tích Mô hình Embedding

- **Google Gemini Embedding** cho Accuracy cao hơn $\approx 4\%$ nhờ khả năng hiểu ngôn ngữ tiếng Việt tự nhiên sâu sắc hơn (được huấn luyện trên tập dữ liệu đa ngôn ngữ khổng lồ).
- **BAAI/bge-m3** (chạy cục bộ) có Latency thấp hơn đáng kể (3.2s so với 4.1s), do không phải gọi API qua mạng, phù hợp cho môi trường không cần API key.
- Cả hai mô hình đều đạt Faithfulness xuất sắc (> 0.90), chứng tỏ ChromaDB truy xuất tốt các đoạn tài liệu có chứa thông tin cần thiết.

5.6.3.3 Khuyến nghị Cấu hình Tối ưu

Kết luận Benchmark – Cấu hình Tối ưu

Cấu hình **Recursive Chunking + Google Gemini Embedding + Gemini 2.5 Flash LLM** cho hiệu năng tốt nhất về Accuracy (0.89) và Faithfulness (0.94), với Latency 4.1 giây – nằm trong giới hạn chấp nhận được (< 5 giây theo NFR-01).

Nếu ưu tiên tốc độ phản hồi, **Recursive + BAAI/bge-m3** là lựa chọn thay thế với hiệu năng gần tương đương nhưng Latency thấp hơn 22%.

5.7 Tóm tắt Kết quả Kiểm thử

Table 5.7: Tóm tắt kết quả kiểm thử toàn bộ hệ thống

Hạng mục kiểm thử	Tổng TC	Pass	Tỷ lệ	
Kiểm thử chức năng	20	20	100%	
Kiểm thử phân quyền	4	4	100%	
Kiểm thử xử lý lỗi	6	6	100%	
Kiểm thử hiệu năng	4	4	100%	
RAGAS Benchmark (Faithfulness ≥ 0.85)	4 configs	4	100%	
Tổng cộng	38	38	100%	

5.7.1 Đánh giá Tổng kết

Thông qua quá trình kiểm thử toàn diện, nhóm đã thu thập được các kết luận sau:

- **Hệ thống đáp ứng 100% Test Cases:** Toàn bộ 38 test cases đều Pass, khẳng định hệ thống hoạt động đúng theo đặc tả SWR.
- **Chất lượng AI vượt ngưỡng yêu cầu:** Cấu hình tốt nhất đạt Faithfulness = 0.94 > 0.85 (NFR-05) và Relevancy = 0.88 > 0.80 (NFR-06).
- **Hiệu năng đạt yêu cầu:** Latency trung bình 3.2 – 4.3 giây, nằm trong giới hạn < 5 giây (NFR-01).
- **Hệ thống gần như loại bỏ hoàn toàn Hallucination:** Faithfulness > 0.90 chứng tỏ AI chỉ trả lời dựa trên tài liệu được cung cấp, không tự bịa đặt thông tin.
- **Benchmark là công cụ nghiên cứu giá trị:** Module Benchmark cung cấp số liệu định lượng khách quan, thay thế hoàn toàn việc đánh giá cảm quan.

5.8 Kết luận Chương

Chương này đã trình bày đầy đủ tài liệu kiểm thử (SWT) bao gồm: phạm vi kiểm thử (In-scope/Out-of-scope), chiến lược và mức độ kiểm thử, 20 Test Cases chức năng chi tiết, cấu hình môi trường Benchmark RAGAS, kết quả so sánh 4 cấu hình RAG với biểu đồ Radar và phân tích chuyên sâu, cùng bảng tóm tắt kết quả 100% Pass. Kết quả kiểm thử khẳng định hệ thống Chatbot RAG đã đáp ứng đầy đủ và vượt trội các yêu cầu chức năng và phi chức năng được đặc tả trong SWR.

CHAPTER 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả Đạt được

Sau 16 tuần nghiên cứu, thiết kế và triển khai tuân thủ nghiêm ngặt quy trình quản lý dự án phần mềm, nhóm đã xây dựng thành công **Hệ thống Chatbot Hỗ trợ Học tập ứng dụng mô hình Retrieval-Augmented Generation (RAG)**. Hệ thống không chỉ cho phép người dùng cuối (sinh viên) đặt câu hỏi và nhận câu trả lời chính xác, minh bạch từ tài liệu môn học, mà còn cung cấp môi trường Sandbox và Benchmark chuyên sâu cho quản trị viên thực hiện đo lường, phân tích và tối ưu hóa các chiến lược xử lý AI.

Các mục tiêu cốt lõi của đề án đều đã được hoàn thành trọn vẹn:

- **Về mặt Kiến trúc và Phần mềm:** Nhóm đã phát triển thành công Backend bằng FastAPI đảm bảo khả năng xử lý I/O bất đồng bộ hiệu suất cao, kết hợp với giao diện Frontend ReactJS hiện đại, Responsive. Quá trình phát triển đã tuân thủ chuẩn mô hình Client-Server nhiều lớp (N-Tier).
- **Về mặt Xử lý AI và RAG Pipeline:** Hệ thống đã triển khai thành công Ingestion Pipeline hoàn chỉnh (bóc tách PDF, làm sạch văn bản, phân mảnh Chunking, và nhúng Embedding) tích hợp trực tiếp vào cơ sở dữ liệu Vector (ChromaDB) để lưu trữ và truy xuất siêu tốc. Luồng RAG được tối ưu thông qua việc kết hợp Prompt Engineering nghiêm ngặt với sức mạnh của LLM (Gemini 2.5 Flash).
- **Về mặt Kiểm thử và Đánh giá:** Việc xây dựng thành công module Benchmark tự động và ứng dụng bộ tiêu chuẩn RAGAS đã cung cấp góc nhìn định lượng khách quan. Kết quả kiểm thử khẳng định hệ thống đạt độ chính xác (Accuracy) 0.89 và độ trung thực (Faithfulness) 0.94, gần như loại bỏ hoàn toàn hiện tượng “ảo giác” (Hallucination) của AI truyền thống.
- **Về mặt Quy trình Kỹ thuật Phần mềm:** Đề án đã được thực hiện bằng một quy trình chuyên nghiệp, từ việc quản lý tiến độ bằng Agile Scrum trên Jira, áp dụng chiến lược Git Flow trên GitHub, cho đến việc biên soạn tài liệu đặc tả và thiết kế (SWP, SWR, SWD, SWT) đầy đủ bằng LaTeX.

6.2 Đánh giá Ưu và Nhược điểm

6.2.1 Ưu điểm Nổi bật

- **Tính Chính xác Cao & Có Nguồn gốc:** Nhờ cơ chế RAG, mọi câu trả lời đều dựa trên các đoạn tài liệu có thật, hệ thống luôn trích dẫn nguồn (tên file, số trang), giúp sinh viên dễ dàng đối chiếu và học tập hiệu quả.
- **Khả năng Đánh giá Định lượng:** Hệ thống tích hợp sẵn công cụ Benchmark chuyên nghiệp, cho phép người dùng thay vì đánh giá cảm quan, có thể sử dụng các chỉ số Toán học/AI (RAGAS) để chọn cấu hình RAG (Chunking, Embedding) tối ưu nhất cho từng loại tài liệu.
- **Kiến trúc Tách biệt & Mở rộng (Decoupled Architecture):** Việc thiết kế giao diện (Interface Pattern) cho Mô hình AI giúp hệ thống dễ dàng kết nối với các LLM mới (GPT-4, Claude) hoặc các mô hình Embedding nội bộ (như BAAI/bge-m3) mà không cần viết lại mã nguồn cốt lõi.

6.2.2 Hạn chế Còn tồn đọng

Mặc dù đạt được nhiều kết quả khả quan, hệ thống ở giai đoạn hiện tại vẫn còn một số điểm giới hạn cần được cải thiện:

- **Xử lý Dữ liệu Đa phương tiện (Multimodal):** Ingestion Pipeline hiện tại chỉ làm việc tốt với văn bản thô (Plain Text). Đối với các tài liệu chứa nhiều biểu đồ, hình vẽ kỹ thuật, hoặc công thức toán học phức tạp, chất lượng trích xuất (OCR) chưa đạt mức tối ưu nhất, làm suy giảm một phần ngữ cảnh của RAG.
- **Quản lý Định danh (Authentication & Authorization):** Tính năng đăng nhập, phân quyền cá nhân hóa và quản lý phiên hội thoại chi tiết cho hàng ngàn sinh viên vẫn còn sơ khai, chưa tích hợp hệ thống xác thực tập trung (như OAuth 2.0 hoặc SSO của trường).
- **Môi trường Triển khai (Deployment):** Hệ thống mới chỉ chạy tốt trên môi trường cục bộ (Localhost) và máy chủ thử nghiệm nhỏ. Việc đóng gói bằng Docker và triển khai lên hạ tầng Cloud (AWS/GCP) để hỗ trợ khả năng tự động mở rộng (Auto-scaling) chưa được thực hiện.

6.3 Định hướng Phát triển Tương lai

Từ những kết quả và hạn chế đã nêu, nhóm đề xuất các hướng nâng cấp thiết thực cho sản phẩm trong tương lai:

1. **Tích hợp AI Xử lý Đa phương tiện (Vision LLM):** Mở rộng Ingestion Pipeline bằng cách sử dụng các mô hình Vision-Language (như Gemini 1.5 Pro Vision) để phân tích, mô tả nội dung của biểu đồ, hình ảnh và công thức Toán học có trong file PDF, sau đó chuyển đổi thành Text Embedding để đưa vào ChromaDB.
2. **Triển khai Cloud & Microservices:** Chuyển đổi kiến trúc Backend hiện tại sang dạng Microservices, đóng gói toàn bộ hệ thống bằng Docker. Triển khai lên nền tảng Kubernetes (K8s) trên AWS hoặc Google Cloud nhằm đảm bảo khả năng chịu tải cao trong các mùa thi cuối kỳ khi lượng sinh viên truy cập tăng vọt.
3. **Cá nhân hóa Lộ trình Học tập (Personalized Learning):** Xây dựng hệ thống tài khoản hoàn chỉnh với OAuth 2.0. Hệ thống sẽ theo dõi lịch sử hỏi đáp của từng sinh viên, phân tích các khái niệm sinh viên thường hỏi hoặc hiểu sai, từ đó tự động gợi ý lộ trình ôn tập và bài tập trắc nghiệm phù hợp.
4. **Phát triển Ứng dụng Di động (Mobile App):** Chuyển đổi từ nền tảng Web-only sang phát triển thêm ứng dụng di động bằng React Native hoặc Flutter, cung cấp trải nghiệm tiện lợi hơn cho sinh viên có thể tra cứu nhanh trên điện thoại thông minh.

6.4 Tổng kết

Đồ án “Xây dựng Hệ thống Chatbot Hỗ trợ Học tập ứng dụng RAG” đã hoàn thành xuất sắc các mục tiêu đề ra. Sản phẩm không chỉ đáp ứng yêu cầu của một công cụ hỏi đáp tự động có độ tin cậy cao, mà còn cung cấp một nền tảng nghiên cứu Benchmark chuyên sâu về AI xử lý ngôn ngữ. Việc phát triển đồ án cũng là một minh chứng thực tiễn về ứng dụng thành công các quy trình công nghệ phần mềm hiện đại (từ Agile, System Design, đến Testing) vào một bài toán trí tuệ nhân tạo có tính ứng dụng cao trong lĩnh vực Giáo dục.